

ZSA Guide

Zcash Shielded Assets: issuance, transfer, and the counterfeiting boundary

m@rek.onl

Abstract

Assuming the Orchard protocol of the *Ironwood Guide* and transaction construction from the *Wallet Guide*, this volume of *The Zcash Arboretum* documents Zcash Shielded Assets: per-asset value bases and asset identity, transparent issuance and finalization (ZIP-227), shielded transfer and burn (ZIP-226), and the split-note construction that guards the counterfeiting boundary. The upstream ZIPs are in flux — their transaction carrier was withdrawn and their fee terms removed upstream — so every claim is classified as specified, designed-but-unspecified, or open. Where ZIP text, fork text, and the implementation disagree, all three readings are stated and the divergence is flagged.

Contents

1	Scope, status, and sources	4
1.1	Scope and deferrals	4
1.2	Status	4
1.3	Sources	5
1.4	Method: the three bins for ZSA	5
2	Asset identity	6
2.1	The issuance key pair	6
2.2	From description to identifier	8
2.3	From identifier to base point	9
2.4	The native asset	10
2.5	Identity on the wire	11
3	Issuance and finalization (ZIP-227)	12
3.1	Issuance keys and the issuer identifier	12
3.2	Asset identity: description, digest, and Asset Base	13
3.3	Issue notes and the derived ρ	13
3.4	The issuance bundle and its encoding	14
3.5	Authorization: BIP-340 over the transaction sighash	15
3.6	Transaction digests	16
3.7	Global issuance state and the consensus rules	17
3.8	Finalization	20
3.9	Fees	20
4	Transfer and burn (ZIP-226)	21
4.1	One new field: the OrchardZSA note	21
4.2	The note commitment: a case split	22
4.3	Per-asset value: the variable-base value commitment	22
4.4	Burn	23
4.5	Split Inputs	24
4.6	The Action statement, extended	24
4.7	The carrier: transaction format, digests, and fees	26
5	Split notes and the counterfeiting boundary	27
5.1	The counterfeit construction the boundary excludes	27
5.2	The split input	28
5.3	Membership as well-formedness: the issuance anchor	29
5.4	The specified circuit delta	30
5.5	The implemented gate	30
5.6	The split nullifier	32
5.7	Ownership, in fact and in prose	33
5.8	Builder mechanics: padding, per asset	34
6	Transaction integration and outlook	34
6.1	Experimental and deployed carriers	34
6.2	The experimental wire format (ZIP-230)	35
6.3	Digests: txid, sighash, and authorizing data (ZIP-246)	36
6.4	Transaction-level consensus rules	38
6.5	The experimental fee policy	39
6.6	Implementation status at the pinned commits	39
6.7	Wallet integration	41

6.8 Outlook 41

1 Scope, status, and sources

Zcash Shielded Assets (ZSA) is the proposed extension of Orchard that carries user-issued assets in the shielded pool. Two consensus ZIPs define it: ZIP 226, Transfer and Burn, and ZIP 227, Issuance, both drafts. The combined protocol, OrchardZSA, enables issuance (ZIP 227) and transfer and burn (ZIP 226) of Custom Assets on the Zcash chain; ZIP 227 must only be implemented in conjunction with ZIP 226, and issuance is deliberately transparent — issuance outputs are unencrypted — so that every asset’s circulating supply can be tracked on chain (zip-0226.rst lines 42–43; zip-0227.rst lines 47, 54).

Terminology is ZIP 227’s: an *Asset* is a type of note transferable on the Zcash blockchain; ZEC is the default — and currently the only defined — Asset on mainnet, TAZ on testnet; a *Custom Asset* is any Asset other than ZEC and TAZ (zip-0227.rst lines 35–39).

The design reduces to one alteration of Orchard’s value algebra. The fixed value base V^{Orchard} of the homomorphic Pedersen value commitment (*Ironwood Guide*, §“Value commitments”) is replaced, per Asset, by an *Asset Base* witnessed inside the circuit; the same Asset Base must appear in the old and the new note commitments and as the base of the Action’s value commitment. Under the discrete-log binding assumption for independently derived bases, the binding signature then enforces balance separately per Asset Identifier without revealing which Assets — or how many distinct ones — a transaction transfers (zip-0226.rst lines 59–63). The transparent protocol is unchanged: unshielding is impossible for Custom Assets, burn is the only supply-reduction mechanism and sends to no address, and the Native Asset cannot be burnt (zip-0226.rst lines 206–230).

1.1 Scope and deferrals

This volume develops only the delta over Orchard: the per-asset note type and note commitment; the Asset Base and its derivation from an Asset Identifier; the per-asset value commitment, the burn mechanism, and the burn-aware balance equation; split notes, the mechanism that replaces dummy spends for Custom Assets; the changes to the Action circuit; the issuance layer of ZIP 227 — issuance keys, issue notes, reference notes, and the node-side issuance state — and the note-plaintext extension.

Everything the delta rests on is constructed in earlier volumes and is cited, never re-derived: notes and note commitments (*Ironwood Guide*, §“Notes and note commitments”); value commitments, the bases V^{Orchard} and R^{Orchard} , and the binding signature (*Ironwood Guide*, §“Value commitments and the binding signature”); the Action statement and dummy spends (*Ironwood Guide*, §“The Action statement”, §“Actions, bundles, and dummy notes”); nullifiers (*Ironwood Guide*, §“Nullifiers”); Sinsemilla (*Ironwood Guide*, §“Sinsemilla: hash, commitment, and short forms”); and, at the wallet layer, transaction construction, PCZT, and fees (*Wallet Guide*, §“Transaction construction”, §“Partially created transactions (PCZT)”, §“Fees (ZIP-317)”).

Deployment status is stated once, in §1.2. Subsequent sections construct their objects without requalifying each of them with it.

1.2 Status

ZSAs are not part of NU7. Its deployment draft, ZIP 259, introduces no transaction format and retains versions 5 and 6 without a ZSA carrier. ZIPs 226 and 227 remain draft proposals for a future upgrade; their mechanisms are not rules an NU7 validator enforces.

The essential distinction is between a cryptographic construction and its *carrier*: the transaction fields that encode it, together with the digests signed by its participants. ZIP 226 assumes the recoverability changes of ZIP 2005, but its carrier references the withdrawn ZIPs 230 and 246. The deployed version 6 format is ZIP 229, which does not carry ZSAs. The extensible version 7

draft, ZIP 248, lists potential ZSA bundles without specifying their encodings or digests. The experimental format below is therefore not a standardised ZSA transaction format.

Two other assignments remain incomplete: ZIP 226 leaves the note-plaintext lead byte unsigned, and the conventional fee rule in ZIP 317 defines no ZSA issuance or asset-creation terms. The fork-specific choices below are identified where used; neither is silently promoted to consensus.

1.3 Sources

ZIPs 226 and 227 define the proposed constructions. The experimental implementation is not identical to those texts; Table 1 fixes which versions supply the fork-specific formulas below. Current upgrade scope follows ZIP 259, not the experimental code’s feature names.

Artifact	Pin / date	Role
zcash/zips (upstream)	69610984, 2026-07-09	primary ZIP ground truth
QED-it/orchard, branch zsa1	cf801a5d, 2026-06-29	crate orchard 0.14.0
QED-it/librustzcash, branch zsa1	3f9b53fc, 2026-04-17	v6 carrier, fee rule
QED-it/zips, branch zsa1	fd71419a, 2026-02-11	fork baseline for diffing

Table 1: Primary ZSA sources, pinned.

Upstream versus fork. Upstream ZIP 226 diverged from the fork text at fd71419a in three substantive ways: it added the ZIP 2005 dependency paragraph; it changed the split-note ψ^{nf} from “sampled uniformly at random on $\mathbb{F}_{p_{\text{Pallas}}}$ ” to a PRFexpand derivation whose domain-separator byte 0x0A is reserved by ZIP 2005; and it replaced the lead byte 0x03 with the $\{\text{ZSALEADBYTE}\}$ placeholder. ZIP 227 upstream differs from the fork only in punctuation and one reference title.

Code versus ZIP. Where the pinned code disagrees with the pinned ZIP text, this volume presents the ZIP formula and flags the divergence inline. Two content divergences head the list. First, the split-note nullifier randomizer: ZIP 226 derives ψ^{nf} with domain byte 0x0A, while orchard-zsa @ cf801a5d computes it with PRFexpand domain 0x09 over a random per-note seed (src/circuit.rs lines 316–319) — the older fork-era derivation. Second, the note-plaintext lead byte: a placeholder upstream, but 0x03 in code (src/primitives/zcash_note_encryption_domain.rs lines 46–49) — the very value ZIP 2005 now specifies differently. A third divergence is one of status rather than content: the digest personalisations in orchard-zsa match the withdrawn ZIP 246 exactly, and librustzcash-zsa implements TxVersion::V6 with the withdrawn ZIP 230 wire format — the entire carrier layer the code implements is specified only by withdrawn ZIPs (§1.2). Further code-versus-text divergences confirmed at the same pins — the omitted explicit-fee digest field, the missing memo branch and its non-spec substitute digest, the empty-action issuance rule, and the valueBurn bound wording — are inventoried in §6.6, §3.7, and §4.4.

The two library pins are separate snapshots, not a single deployed stack. Checks of the implementation below establish library behaviour, not node-side enforcement of ZIP 227’s issuance state.

1.4 Method: the three bins for ZSA

This volume classifies every design-stage claim as *specified*, *designed but unspecified*, or an *open problem*, and keeps the bin visible in the prose. Bin (i) holds the constructions and consensus rules that the live ZIP texts state directly: the per-asset note and commitment, Asset Base derivation, value commitments and balance equation, the split-note circuit delta, issuance keys, issue

and reference notes, the issuance state machine, and burn. Bin (ii) holds both the gate-level realization of that circuit delta and the carrier layer: the v6 wire format, digest tree and sighash, note-plaintext lead byte, and ZSA fee terms. Those details are fixed only by implementation, an externally hosted unversioned circuit description, withdrawn ZIPs, or the QED-it fork texts (§1.2). Bin (iii) holds assignment to a future network upgrade and activation, the eventual lead-byte assignment, and the carrier format’s future home. Formulas below identify whether their source is ZIP text, pinned code, or an explicitly labelled derivation in this volume.

2 Asset identity

An OrchardZSA note denominates value in one of many assets sharing a single shielded pool, so the protocol must attach to every note an asset identity satisfying two demands. First, uniqueness: “The Asset identification should be unique (among all shielded pools) and different issuer public keys should not be able to generate the same Asset Identifier” (zip-0227.rst:76). Second, usability inside the circuit: the identity must ultimately be a Pallas point, because it serves as the asset-specific base of the value commitment (§2.4). ZIP 227 meets both demands with one derivation chain,

$$\begin{aligned} \text{asset_desc} &\rightarrow \text{assetDescHash}; \\ (\text{issuer}, \text{assetDescHash}) &\rightarrow \text{AssetId} \rightarrow \text{AssetDigest} \rightarrow \text{AssetBase}, \end{aligned}$$

which the ZIP itself presents as a relation diagram (zip-0227.rst:210–224): the issuer-chosen description string is hashed, the hash paired with the issuer key into an identifier, the identifier hashed into a digest, and the digest mapped onto the Pallas curve. Every Asset issued under ZIP 227 is scoped to the issuer that issued it. Its Asset Identifier is intended to be globally unique and is “used across all Zcash protocols that support ZSAs” (zip-0227.rst:227–240); for two descriptions under one issuer, uniqueness is computational and rests on BLAKE2b-256 collision resistance.

This section constructs the chain end to end: the issuance key pair at its root (§2.1); the hash steps from description to identifier (§2.2); the group hash from identifier to base point (§2.3); the native asset’s special position outside the chain (§2.4); and the parts of the chain consensus sees on the wire versus recomputes (§2.5). Deployment status for the whole protocol is fixed once, in §1.2, and not restated per object.

2.1 The issuance key pair

The OrchardZSA protocol adds exactly two keys to the key material of Orchard: the *issuance authorizing key* isk, which authorizes the issuance of Asset Identifiers by a given issuer and is used only by that issuer, and the *issuance validating key* ik, which validates issuance transactions (zip-0227.rst:84–91). Neither key touches the spend path. The ZIP’s rationale: the issuance key structure is independent of the original key tree but derived analogously via ZIP 32, which keeps issuance details and Asset Identifiers consistent across multiple shielded pools, and separates issuance authority from spend authority — allowing a potential transfer of issuance authority without compromising spend authority (zip-0227.rst:524).

Definition 2.1 (Issuance authorization signature scheme). The scheme IssueAuthSig is instantiated as a BIP-340 Schnorr signature over the secp256k1 curve, with constants set per the secp256k1 standard parameters as described in BIP 340. Its associated types are: messages are arbitrary byte sequences; signatures are 64-byte sequences or $\perp$; public keys are 32-byte sequences or $\perp$; private keys are 32-byte sequences (zip-0227.rst:122–135). Signing fixes the BIP-340 auxiliary data to $a = [0x00]^{32}$: the signature is $\sigma := \text{Sign}(\text{isk}, M)$ with auxiliary data a ,

or $\perp$ if Sign fails; its scheme-level encoding is $\text{issueAuthSig} = [0x00] \parallel \sigma$ (zip-0227.rst:190–207).

In the implementation, type ZSASchnorr realizes the scheme with `ALGORITHM_BYTE = 0x00`, secret keys as `secp256k1::SecretKey`, public keys as `XOnlyPublicKey`, and signing via `sign_schnorr_with_aux_rand` with the all-zero 32-byte auxiliary array.

Construction 2.2 (Issuance key derivation). The issuance authorizing key is generated by ZIP 32’s hardened-only key derivation — the maps MKGh and CKDh of ZIP 32, “Specification: Hardened-only key derivation”, whose Orchard instance is the *Wallet Guide*’s §“Hardened-only derivation” — instantiated in a dedicated Issuance context (zip-0227.rst:139–166). Let S be a seed of at least 32 and at most 252 bytes with at least 256 bits of entropy, as ZIP 32’s “Specification: Wallet seeds” requires.

- (1) *Context*. The issuance context sets `Issuance.MKGDomain = ZcashSA_Issue_V1` and `Issuance.CKDDomain = 0x81` (zip-0227.rst:148–149).
- (2) *Master key*. Set $m_{\text{Issuance}} := \text{MKGh}^{\text{Issuance}}(S)$.
- (3) *Child derivation*. Set $\text{CKDsk}((\text{sk}_{\text{par}}, c_{\text{par}}), i) := \text{CKDh}^{\text{Issuance}}((\text{sk}_{\text{par}}, c_{\text{par}}), i)$. The two-argument call fixes the general form’s *lead* = 0 and *tag* = [], exactly the case whose suffix the ZIP-32 encoding omits (zip-0032.rst:449–462).
- (4) *Path*. Derive along m_{Issuance} / *purpose'* / *coin_type'* / *account'*, with *purpose* fixed to 227 (0xe3; hardened 0x800000e3 per BIP 43), *coin_type* as in the ZIP 32 key path levels, and *account* fixed to index 0 (zip-0227.rst:160–164).
- (5) *Key extraction*. From the generated pair (sk, c) , set $\text{isk} := \text{sk}$ (zip-0227.rst:164–166).

The implementation realizes the context as a struct `Issuance` implementing the zip32 crate’s hardened-only Context trait, with `MKG_DOMAIN` set to `ZcashSA_Issue_V1` and with `CKD_DOMAIN` reusing the Orchard byte, constant `PrfExpand::ORCHARD_ZIP32_CHILD`; the seed-level constructor `from_zip32_seed` derives along `[227', coin_type', 0']` and rejects any non-zero account with error `NonZeroAccount`. The zip32 crate 0.2.0 implements MKGh and CKDh exactly, and its `derive_child(index)` calls `ckdh_internal(index, 0, [])` — *lead* = 0, *tag* = [], omitted from the PRF message.

Remark 2.3 (Domain separation between the Orchard and Issuance trees). The byte `Issuance.CKDDomain = 0x81` equals `Orchard.CKDDomain` (zip-0032.rst:476), and the implementation reuses `PrfExpand::ORCHARD_ZIP32_CHILD` (zcash_spec src/prf_expand.rs:119 @ d5e84264). Separation between the two ZIP-32 trees therefore rests entirely on the differing MKGDomain: distinct master secret keys and chain codes feed every subsequent PRFexpand call. Neither ZIP states whether the byte reuse is deliberate; ZIP 32 asks only that CKDDomain values be tracked as part of the global set of domains defined for PRFexpand. The derivation is *specified*; we flag the shared byte because the separation argument is easy to misattribute to it.

Definition 2.4 (Issuance validating key and issuer identifier). The issuance validating key is $\text{ik} := \text{PubKey}(\text{isk})$, where `PubKey` is the BIP-340 algorithm and the output is in big-endian order as defined in BIP 340; the derivation returns $\perp$ if `PubKey` fails, which happens with very low probability, in which case the keys are discarded and derivation repeated with a different isk (zip-0227.rst:168–183). Its encoding prepends a signature-scheme byte, which MUST

be 0x00, indicating BIP 340:

$$\text{ik_encoding} = [0x00] \parallel \text{ik} \quad (33 \text{ bytes}),$$

and the issuer identifier is defined by $\text{issuer} = \text{ik_encoding}$ (zip-0227.rst:103–110, 178–179). The encoding currently appears on chain only in the issuer field of an issuance bundle (zip-0227.rst:178–180).

Remark 2.5 (Permanence of the issuer binding). ZIP 227 notes that “the equality of issuer and ik_encoding might not hold in future when key rotation is specified for issuance key pairs” (zip-0227.rst:103–110). No rotation mechanism exists today: an asset’s identity is permanently bound to the one ik under which it was issued, and the ZIP’s remedy for key compromise is to finalize the asset and reissue under a new Asset Identifier: “If an issuer identifier is compromised, the finalize boolean for all the Assets issued with that key should be set to 1; then the issuer should change to a new issuance authorizing key (hence a new issuer identifier), and issue new Assets, each with a new AssetId” (zip-0227.rst:640–645, Security and Privacy Considerations, “Issuance Key Compromise”). This is a *stated limitation* of the specified design; the rotation mechanism itself is an open design area.

2.2 From description to identifier

An issuer names each of its assets with an *asset description* asset_desc : a non-empty byte sequence, which SHOULD be a well-formed UTF-8 code unit sequence according to Unicode 15.0.0 or later (zip-0227.rst:237–241). Within the scope of an issuer, Asset Identifier uniqueness is obtained by way of the asset description (zip-0227.rst:237–240). The current ZIP imposes no maximum length — its rationale notes that a description “can be longer than 512 bytes without incurring chain costs”, the 512-byte cap being an earlier design (zip-0227.rst:528–539). The description is also global: ZIP 226 states that “Every Asset is defined by an Asset description, asset_desc , which is a global byte string (scoped across all future versions of Zcash)”, so that in a future upgrade the same string can carry over to a different shielded pool, with nodes transforming the identifier chain between pools without balance violations (zip-0226.rst:93–103).

Definition 2.6 (Asset Identifier). The description is hashed,

$$\text{assetDescHash} := \text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \text{asset_desc})$$

(zip-0227.rst:243–247), and the Asset Identifier is the pair

$$\text{AssetId} := (\text{issuer}, \text{assetDescHash})$$

(zip-0227.rst:249–253), with the canonical encoding

$$\text{EncodeAssetId}(\text{AssetId}) := [0x00] \parallel \text{issuer} \parallel \text{assetDescHash} \quad (66 = 1 + 33 + 32 \text{ bytes}),$$

where the initial 0x00 is a version byte enabling future issuance protocols (zip-0227.rst:255–261). The version byte is not to be confused with the byte that follows it — the first byte of issuer, which indicates the signature scheme (Definition 2.4); every current encoding happens to begin with two 0x00 bytes of distinct meaning.

The pair structure carries the uniqueness argument. Because the issuer identifier is a component of AssetId, two distinct issuer-key encodings cannot produce the same pair; in the ZIP’s words, the Custom Asset “is described via a combination of the issuer identifier and an asset description string, to preclude the possibility of two different issuers creating colliding Custom Assets” (zip-0227.rst:525). Within one issuer, distinct descriptions separate assets via

assetDescHash only subject to BLAKE2b-256 collision resistance; for every new Asset there MUST be a new and unique Asset Identifier (zip-0226.rst:93–96).

Remark 2.7 (Why a hash, not the string). ZIP 227 replaces the direct description string of an earlier design with a collision-resistant hash for five stated reasons (zip-0227.rst:528–559): a fixed 32 bytes per Issue Action costs less bandwidth than up to 512 bytes; descriptions can exceed 512 bytes without chain cost; carrying the byte string on chain would not make the user-visible description consensus-visible anyway (the string could itself be a hash of an off-chain description); absent key rotation, marking an issuer as trusted is insufficient — each Asset Identifier needs independent out-of-band verification that conveys the description; and an on-chain description is an “attractive nuisance” — hashing forces wallets to obtain the description from a trusted party, a registry, or direct user approval. Consistent with the last point, wallets MUST NOT display just the `asset_desc` string as the name of the Asset; the ZIP suggests a petname system mapping names to Asset Identifiers via client configuration, or the Asset Digest as a compact identifying byte sequence, e.g. in QR codes (zip-0227.rst:263–266).

Implementation. The identifier is the enum variant `AssetId::V0`, holding the validating key and the 32-byte description hash; `encode_asset_id` emits the version byte `0x00`, the 33-byte key encoding, and the hash — 66 bytes, matching `EncodeAssetId`. The function `compute_asset_desc_hash` enforces non-emptiness through its argument type `NonEmpty<u8>` and *panics* if the description is not well-formed UTF-8 (orchard-zsa src/issuance.rs:135–154 @ cf801a5d). The specification says SHOULD; the implementation enforces a hard MUST at its API boundary. Since only the 32-byte hash is consensus-visible, neither variant is consensus-enforceable: we classify the UTF-8 constraint as an issuer-side convention — *specified* as a SHOULD, with the stricter behaviour an implementation choice.

2.3 From identifier to base point

Two further steps land the identifier on the Pallas curve. The full chain, collected:

$$\begin{aligned} \text{assetDescHash} &:= \text{BLAKE2b-256}(\text{ZSA-AssetDescCRH}, \text{asset_desc}), \\ \text{AssetId} &:= (\text{issuer}, \text{assetDescHash}), \\ \text{AssetDigest}_{\text{AssetId}} &:= \text{BLAKE2b-512}(\text{ZSA-Asset-Digest}, \text{EncodeAssetId}(\text{AssetId})), \\ \text{AssetBase}_{\text{AssetId}} &:= \text{ZSAValueBase}(\text{AssetDigest}_{\text{AssetId}}), \end{aligned}$$

where the digest is defined at zip-0227.rst:268–278, the base at zip-0227.rst:280–290, and the OrchardZSA instantiation of the final step is

$$\text{ZSAValueBase}(\text{AssetDigest}) := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{OrchardZSA}, \text{AssetDigest})$$

(zip-0227.rst:292–301). The subscript `AssetId` may be omitted when clear from context (zip-0227.rst:224). The group hash is the protocol specification’s `GroupHashP` (§5.4.9.8, “Group Hash into Pallas and Vesta”), whose construction — `expand_message_xmd` over BLAKE2b, two field elements through the simplified SWU map, summed and carried through the isogeny — the *Ironwood Guide* develops in “GroupHash, domain separation, and nothing-up-my-sleeve generators”; we reuse it as a deterministic, public, nothing-up-my-sleeve map into the Pallas group and do not re-derive it here.

ZIP 226 types the result as a *non-trivial* point: `AssetBase` is “the unique element of the Pallas group that identifies each Asset in the Orchard protocol ... a valid group element that is not the identity and is not $\perp$ ” (zip-0226.rst:111–117), a non-identity point of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ in our notation. Here uniqueness can only mean that the derivation is deterministic for a given identifier, not that the map is injective: a 512-bit digest is mapped into a roughly 255-bit group, so collisions exist in

principle. Asset separation at the base and global-state levels is computational and requires that collisions in the digest/group-hash chain be infeasible to find. Its byte form is

$$\text{asset_base} := \text{LEBS2OSP}_{\ell_{\mathbb{P}}}(\text{repr}_{\mathbb{P}}(\text{AssetBase})),$$

the 32-byte representation stored in note plaintexts and burn fields (zip-0226.rst:114; zip-0230.rst:540–545). ZIP 226 notes the encoding assumes canonicity, “true for the Pallas group, but may not hold for future shielded protocols” (zip-0226.rst:111–117). Table 2 collects the sizes along the chain.

Object	Bytes	Form	Source
ik	32	x-only BIP-340 key, big-endian	zip-0227.rst:130–133
ik_encoding = issuer	33	[0x00] ik	zip-0227.rst:178–179
assetDescHash	32	BLAKE2b-256 output	zip-0227.rst:243–247
EncodeAssetId(AssetId)	66	[0x00] issuer assetDescHash	zip-0227.rst:255–259
AssetDigest	64	BLAKE2b-512 output	zip-0227.rst:268–278
asset_base	32	LEBS2OSP $_{\ell_{\mathbb{P}}}$ (repr $_{\mathbb{P}}$ (·))	zip-0230.rst:540–545

Table 2: Byte sizes along the asset-identity derivation chain.

Why the base must be a genuine group-hash output. The chain’s final step is not a formality. In a transfer, the output note of a split Action cannot be allowed to carry an arbitrary Pallas point as its base: “We must enforce it to be an actual output of a GroupHash computation ... Without this enforcement the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds” (zip-0226.rst:299–307). The circuit therefore ensures the value base points of all output notes are actual GroupHash outputs “as they originate in the Issuance protocol which is publicly verified”, and ZIP 226 adds: “We prevent a potential malleability attack on the Asset Identifier by ensuring the output notes receive an Asset Base that exists on the global state” (zip-0226.rst:104–106, 299–307). The enforcement mechanism belongs to the transfer sections of this volume (§5).

Remark 2.8 (The identity-point corner case). The protocol specification’s $\text{GroupHash}^G(D, M)$ has the full group as codomain, so its output can in principle be the zero point (protocol.tex §5.4.9.8; the *Ironwood Guide*, §“GroupHash, domain separation, and nothing-up-my-sleeve generators”, constructs the hash-to-curve pipeline). ZIP 227’s ZSAValueBase never states what happens in that event; the non-identity requirement comes from ZIP 226’s typing of the note field (zip-0226.rst:111–117) and from the implementation, which panics inside `AssetBase::custom` (“this will happen with negligible probability”), rejects the identity in `AssetBase::from_bytes`, and carries an `Error::AssetBaseCannotBeIdentityPoint` check in `IssueAction::verify` that the panic renders unreachable (orchard-zsa src/note/asset_base.rs:98–138, src/issuance.rs:216–244 @ cf801a5d). Whether the intended consensus behaviour is “reject the transaction” or “the issuer must pick a different description” is unspecified in the ZIP text. The event has negligible probability; the specification gap is nevertheless real, and we classify the corner case as *open*.

2.4 The native asset

ZEC is the default — and currently the only defined — Asset for mainnet, TAZ for testnet; a *Custom Asset* is any Asset other than ZEC and TAZ (zip-0227.rst:35–39). The native asset does not pass through the derivation chain at all. Its base is fixed by definition: “the Asset Base of the ZEC Asset will be kept as the original value base point, V^{Orchard} ” (zip-0226.rst:98); for ZEC,

$$\text{AssetBase}_{\text{AssetId}} := V^{\text{Orchard}},$$

“so that the value commitment for ZEC notes is computed identically to the Orchard protocol deployed in NU5. As such $\text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v)$ as defined in ZIP 224 is used as $\text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(V^{\text{Orchard}}, v)$ ” (zip-0226.rst:182–194). The point $V^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash : Orchard-cv}, "v")$ is already constructed, together with its companion R^{Orchard} , in the *Ironwood Guide*, “Value commitments”, Definition “Net value commitment”; we cite it and do not re-derive it.

Two consequences follow. First, indistinguishability: ZIP 230 states that “An Orchard note is essentially equivalent to an OrchardZSA note with $\text{AssetBase} = V^{\text{Orchard}}$; in particular, they have the same note commitment ... and need not be distinguished in note plaintexts” (zip-0230.rst:531–538), and ZIP 226’s privacy discussion states that OrchardZSA note commitments for the native asset are indistinguishable from those for non-native Assets (zip-0226.rst:83–84). Second, exclusion from burning: the first consensus rule for assetBurn requires $\text{AssetBase} \neq V^{\text{Orchard}}$ for every $(\text{AssetBase}, v) \in \text{assetBurn}$ (zip-0226.rst:220–223) — the native asset cannot be burnt.

Implementation. The native base is $\text{AssetBase}::\text{zatoshi}()$, computed as the Pallas hash-to-curve over personalization “z.cash:Orchard-cv” of the message “v” — exactly V^{Orchard} ; $\text{AssetBase}::\text{is_zatoshi}()$ is a constant-time equality with that point; and $\text{ValueCommitment}::\text{derive}$ computes $[v]V + [\text{rcv}]R$ with $V = \text{asset.cv_base}()$ and R the fixed base R^{Orchard} , so a ZEC note’s value commitment is bit-identical to NU5 Orchard’s.

2.5 Identity on the wire

Of the whole chain, consensus sees exactly two links on the issuance side: the issuer and the description hash. The v6 transaction’s issuance bundle carries issuerLength (compactSize), issuer (the issuer identifier per ZIP 227), nIssueActions (compactSize), vIssueActions , and issueAuthSig — the signature bytes preceded by a 0x00 byte indicating BIP 340. The first four fields are always present; issueAuthSig is present iff $\text{nIssueActions} > 0$, and if $\text{nIssueActions} = 0$ then issuerLength MUST be 0 (zip-0230.rst:166–179, 215–217, 430–443). Each Issue Action carries assetDescHash as byte[32], nNotes (compactSize, possibly zero; for an existing Asset, this permits finalization without further issuance), vNotes (115 bytes per note description: 43-byte recipient, 8-byte value, 32-byte ρ , 32-byte rseed), and flagsIssuance with finalize in the least significant bit; the burn encoding carries assetBase as byte[32] (zip-0230.rst:367–383, 446–494). On the transfer side, the v6 Orchard note plaintext inserts the asset between rseed and the memo key (the 32-byte key that replaces the in-band 512-byte memo under ZIP-231 and from which the recipient derives the key decrypting its memo in the transaction-level memo bundle; §4.7),

$$[\text{leadByte}] \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}}$$

(zip-0230.rst:523–553).

The consensus rules bind identity to validation: the ik_encoding used to validate issueAuthSig MUST be taken from the issuer field and MUST start with byte 0x00, and for each Issue Action, the Issue Note’s AssetBase is derived from the assetDescHash field of the action and the issuer field of the bundle (zip-0227.rst:483–491). The implementation makes the recomputation structural. Function read_bundle decodes the issuer via $\text{IssueValidatingKey}::\text{decode}$, checking the algorithm byte; function read_action reads the 32-byte hash and recomputes the base as $\text{AssetBase}::\text{custom}$ of $\text{AssetId}::\text{new_v0}(\text{ik}, \text{asset_desc_hash})$. The Asset Base is never carried on the wire for issuance, always re-derived. Redundantly with construction, $\text{IssueAction}::\text{verify}$ checks that every note’s base equals the one derived from the validating key and the description hash, failing with error $\text{IssueBundleIkMismatchAssetBase}$ otherwise. The Asset Identifier is therefore not serialized

as a standalone field: its two components are consensus-visible, and the Asset Base is a recomputable function of them, anchored to the issuer key that signs the bundle.

Classification. The entire derivation chain of this section is *specified*: every formula above appears in the ZIP texts at rigor and is realized by the pinned implementation forks at the pinned commits (orchard-zsa @ cf801a5d, librustzcash-zsa @ 3f9b53fc). Both ZIPs are Status Draft, Category Consensus, owned by Pablo Kogan, Vivek Arte, Daira-Emma Hopwood, and Jack Grigg, created 2022-05-01 and discussed in zcash/zips issue #618 and PR #680 (zip-0226.rst:1–18, zip-0227.rst:1–18); deployment status is stated in the scope section. Three reference gaps qualify the bin. First, ZIP 227’s Test Vectors and Reference Implementation sections read “LINK TBD” and its Deployment section “TBD” (zip-0227.rst:661–675); the de-facto assertion-checked vectors live in the fork: file test_vectors/asset_base.rs, generated from the zcash-hackworks test-vectors script orchard_zsa_asset_base.py, is re-played by a test that recomputes the full chain from (ik_encoding, asset_desc) to asset_base (src/note/asset_base.rs:236–261 @ cf801a5d), with companion vectors for the signature scheme and the ZIP-32 derivation in issuance_auth_sig.rs and zip32.rs. The vectors’ fixed [u8; 512] description arrays are a relic of the dropped 512-byte cap and imply no bound. Second, the corner cases flagged above remain: the identity-point behaviour is *open* (Remark 2.8), the UTF-8 constraint is a SHOULD with a stricter implementation (§2.2), and key rotation is an open design area (Remark 2.5). Third, upstream zcash/zips @ 69610984 is the text of record; the QEDit zips fork (zips-zsa @ fd71419a) is essentially identical for the asset-identity material, and the differences — upstream’s later additions of a ZIP-2005 dependency, a nullifier ψ^{nf} derivation, and a lead-byte rule — do not touch this chain.

3 Issuance and finalization (ZIP-227)

The transfer mechanism below moves Custom Assets between shielded notes; this section brings them into existence. ZIP-227 specifies issuance on the Zcash chain: a per-transaction *issuance bundle* in which a publicly named issuer creates new notes, a per-Asset *finalize* switch that ends issuance forever, and a node-maintained global supply map.

Issuance is deliberately *not* shielded. The ZIP’s Motivation states: “This ZIP only enables *transparent* issuance. As a first step, transparency will allow for proper testing of the applications that will be most used in the Zcash ecosystem, and will enable the supply of Assets to be tracked” (ZIP-227, *Motivation*). The issuer, Asset, issued amounts, finalization status, raw recipient addresses, ρ values, and random seeds are serialized in the clear. For an ordinary recipient, the address does not reveal the nullifier key, so a later spend is designed to be unlinkable to the public issue-note description. The mandatory all-zero-key reference note is an explicit exception (§3.3).

Encoding boundary. The issuance mechanism follows ZIP 227. Its encodings and digests below follow the experimental ZIP 230/246 carrier, not ZIP 229’s deployed v6 format (§1.2). Fork-only fee rules and differences between the ZIP and the pinned libraries are marked where they affect a construction.

3.1 Issuance keys and the issuer identifier

Issuance authority is a new capability, deliberately disjoint from spend authority. The rationale in the ZIP: the issuance key tree is independent of the spending key tree but derived analogously (via ZIP 32), so an organization can delegate issuance without exposing funds, and a future scheme can replace the issuance leaf with a multisignature without touching spending keys (ZIP-227, *Rationale*, and *Issuance Key Derivation*).

The key material itself is constructed once, in §2.1: the BIP-340 signature scheme (Definition 2.1), the hardened-only ZIP-32 derivation of isk in the Issuance context (Construction 2.2), and the validating key with the issuer identifier, $\text{ik} := \text{PubKey}(\text{isk})$ and $\text{issuer} = \text{ik_encoding} = [0x00] \parallel \text{ik}$ (Definition 2.4). We restate only the parameters the issuance layer consumes: seed S of 32 to 252 bytes, $\text{Issuance.MKGDomain} = \text{ZcashSA_Issue_V1}$, $\text{Issuance.CKDDomain} = 0x81$, hardened path $m_{\text{Issuance}} / 227' / \text{coin_type}' / 0'$, and $\text{isk} := \text{sk}$ at the leaf (ZIP-227, *Issuance Key Derivation*). No rotation exists today: the on-chain issuer name is the validating key itself, and the ZIP notes the equality might not hold in future if key rotation is ever specified (Remark 2.5; §3.8).

3.2 Asset identity: description, digest, and Asset Base

Every Custom Asset is named by its issuer plus an issuer-chosen description, and consumed by the protocol as a Pallas point. The full derivation chain — assetDescHash , AssetId , EncodeAssetId with its version byte, AssetDigest , and $\text{AssetBase} = \text{ZSAValueBase}(\text{AssetDigest})$ — is constructed once, in §2.2 and §2.3 (Definition 2.6); the hash-not-string rationale is Remark 2.7, and the UTF-8 SHOULD-versus-panic delta between ZIP and implementation is recorded in §2.2. The native Asset bypasses the chain entirely: its base is the Orchard value-commitment base V^{Orchard} (§2.4). ZIP 226 excludes the Pallas identity from the Asset-Base type; $\text{AssetBase}::\text{custom}$ panics after hashing to it, while $\text{AssetBase}::\text{from_bytes}$ rejects it on parse (Remark 2.8). What the issuance layer adds to the chain is consensus-side recomputation: validators re-derive every Issue Note’s AssetBase from the bundle’s issuer and the action’s assetDescHash , never reading it off the wire (§2.5; consensus rules, §3.7).

3.3 Issue notes and the derived ρ

Definition 3.1 (Issue note). An *Issue Note* is a tuple $(d, \text{pk}_d, v, \text{AssetBase}, \rho, \text{rseed})$,

$$\text{Note}^{\text{Issue}} := \{0, 1\}^{\ell_d} \times \text{KA}^{\text{Orchard}}.\text{Public} \times \{0 \dots 2^{\ell_{\text{value}}} - 1\} \times \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})^* \times \mathbb{F}_{p_{\text{Pallas}}} \times \{0, 1\}^{8 \cdot 32},$$

a diversifier and diversified transmission key as in Orchard, a value v with $\ell_{\text{value}} = 64$, a non-identity Asset Base, the nullifier-input field element ρ , and a 32-byte rseed that MUST be sampled uniformly at random by the issuer (ZIP-227, *Issue Note*; the ZIP writes the fourth and fifth components $\mathbb{P}^*$ and $\mathbb{F}_{q_{\mathbb{P}}}$, our $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})^*$ and $\mathbb{F}_{p_{\text{Pallas}}}$).

Issue-note commitments are not serialized: the wire format carries the note fields themselves in the clear (§3.4), and each validator computes the commitment — by the OrchardZSA NoteCommit of §4 — and appends it to the Orchard note commitment tree. For an ordinary issue note, the recipient address does not reveal the nullifier key; under the nullifier PRF assumptions, a later nullifier is computationally unlinkable to that public description. This privacy claim does not apply to the reference note, whose all-zero spending key is public (ZIP-227, *Issue Note*, *Reference Notes*, and *Issuance Protocol*). Issuance is transparent at creation; later transfers of ordinary issue notes are shielded.

Computation of ρ . An Orchard note takes $\rho := \text{nf}_{\text{old}}$ from the Action that creates it (the *Ironwood Guide*, §“Nullifier chaining”); an issue note has no such Action, so ZIP-227 derives ρ from the transaction’s OrchardZSA transfer bundle instead:

$$\text{DeriveIssuedRho}(\text{nf}, i_A, i_N) := \text{ToBase}(\text{PRFexpand}_{\text{nf}}(0x84 \parallel \text{I2LEOSP}_{32}(i_A) \parallel \text{I2LEOSP}_{32}(i_N))),$$

$$\rho := \text{DeriveIssuedRho}(\text{nf}_{0,0}, \text{index}_{\text{Action}}, \text{index}_{\text{Note}}),$$

with the PRF keyed by nf as 32 little-endian bytes (I2LEOSP_{256}), where $\text{nf}_{0,0}$ is the nullifier of the input note in the *first* Action of the *first* Action Group of the OrchardZSA bundle (the v_6

format partitions a bundle’s Actions into one or more Action Groups, each carrying its own anchor and proof; §6.2), and the two indices are the zero-based positions of the Issue Action within the issuance bundle and of the note within its action (ZIP-227, *Computation of ρ*). The domain-separator byte 0x84 extends the PRFexpand lead-byte registry (the *Wallet Guide*, §“The PRFexpand lead-byte registry”); the patched `zcash_spec` carries it as `ORCHARD_DERIVED_ISSUE_RHO` (QED-it `zcash_spec`, `src/prf_expand.rs` line 107 @ d5e8426). The implementation computes the identical expression — PRFexpand keyed by the 32-byte little-endian nullifier over 0x84 followed by two 4-byte little-endian indices, reduced by ToBase. This derivation is why an issuance bundle cannot travel alone: requiring at least one Action Group both supplies $\text{nf}_{0,0}$ and blocks replay — a transaction containing only an issuance bundle would have a SIGHASH computed solely from that bundle, so a duplicated bundle would share the SIGHASH (ZIP-227, *Rationale*).

In the builder, notes are created with ρ unset, via `new_issue_note`; once the transfer bundle’s first nullifier is known, `update_rho_for_issuance_note` sets ρ and resamples `rseed` until both the derived ephemeral key is valid and the note commitment is not $\perp$ ¹ (`orchard-zsa`, `src/note.rs` lines 457–489 @ cf801a5d).

Reference notes. The first time an Asset is issued, the issuer MUST place a *reference note* as the first note of that Issue Action: a note of value $v = 0$ whose recipient (d, pk_d) is the default diversified payment address (diversifier index 0) derived from the *all-zero* Orchard spending key (ZIP-227, *Reference Notes*). The implementation hard-codes the resulting 43-byte raw address as `RAW_REFERENCE_RECIPIENT`, byte-identical to the array in the ZIP, with `ReferenceKeys::sk() = SpendingKey::from_bytes([0; 32])`; the predicate `is_reference_note` requires zero value and exactly this recipient, `get_reference_note` accepts only the *first* note of an action, and the builder inserts the reference note itself when told the issuance is a first issuance — adding a recipient for an Asset that already has notes fails with `CannotBeFirstIssuance`. The reference note is retained in global state as `noteref` (§3.7).

The ZIP mandates reference notes and stores them in consensus state, but states no rationale for their existence. The evident function — that every issued Asset Base is guaranteed at least one note in the commitment tree, available as a ZIP-226 split-input source for a first spend — is our reading, stated nowhere in the ZIP text or its rationale sections; we classify the mechanism itself as *specified* and this purpose as *designed but unspecified*.

3.4 The issuance bundle and its encoding

An *Issuance Bundle* carries the issuer identifier, the list of Issue Actions, and one authorizing signature: `issuer`, which lets validators verify that each `AssetId` is properly associated with this issuer; `vIssueActions`, an array of Issue Actions; and `issueAuthSig`, the encoding of a signature over the transaction SIGHASH by the issuance authorizing key `isk` (ZIP-227, *Issuance Bundle*). An *Issuance Action* groups the notes issued for one Asset: the 32-byte `assetDescHash`, the note list, and a flags byte carrying `finalize` (ZIP-227, *Issuance Action*). Table 3 gives the v6 serialization; the encoding rigor caveat of the section introduction applies.

Three encoding facts deserve emphasis. First, the `recipient` field is exactly an Orchard raw payment address (the *Wallet Guide*, §“Raw encodings”), and together with the bundle’s issuer and the action’s `assetDescHash`, the four fields of an `IssueNoteDescription` determine the Issue Note completely (ZIP-230, *Issue Note Description*; ZIP-227, *Issue Note*) — the note commitment is deliberately absent, since validators recompute it (§3.3). Second, the encoding permits `nNotes = 0`; for an existing Asset this lets the issuer finalize without issuing more (ZIP-230, *Issuance Action Description*; the new-Asset corner case is recorded in §3.7). Third, the flags byte is strict: `IssuanceFlags::from_byte` returns `None` whenever any bit other than the LSB is set, and deserialization fails on such bytes, enforcing ZIP-230’s “remaining bits are set to 0” at parse time.

¹Strictly, these negligible-probability rejection steps condition `rseed`, a technical deviation from the ZIP’s “sampled uniformly at random” (ZIP-227, *Issue Note*).

Field	Type	Notes
issuerLength	compactSize	MUST be 0 if nIssueActions = 0
issuer	byte[issuerLength]	= ik_encoding, 33 bytes when present
nIssueActions	compactSize	
vIssueActions	IssueAction[nIssueActions]	
issueAuthSig	IssueAuthSignature	present iff nIssueActions > 0
<i>Per IssueAction:</i>		
assetDescHash	byte[32]	
nNotes	compactSize	may be 0 (existing-Asset finalization)
vNotes	IssueNoteDescription[nNotes]	115 bytes each
flagsIssuance	byte	finalize at LSB; other bits MUST be 0
<i>Per IssueNoteDescription (43 + 8 + 32 + 32 = 115 bytes):</i>		
recipient	byte[43]	LEBS2OSP ₈₈ (d) pk _d [*]
value	uint64	little-endian
rho	byte[32]	as for Orchard notes
rseed	byte[32]	as for Orchard notes

Table 3: ZSA issuance fields of the v6 transaction format (fork ZIP-230, *Transaction Format, Issuance Action Description*, and *Issue Note Description* @ zips-zsa fd71419; upstream text identical apart from the Withdrawn status).

The `librustzcash-zsa` serializer pins down the absent-bundle corner cases: `read_bundle` reads `issuer` as a length-prefixed vector and returns `None` only for an empty issuer with zero actions, erroring on an empty issuer with non-empty actions and on a non-empty issuer with zero actions; `write_bundle` encodes an absent bundle as two zero `compactSizes`. The authorization serializes as a length-prefixed `sighashInfo` followed by the length-prefixed signature encoding, and the parser accepts exactly `[0x00]` as `sighashInfo` (`transaction/components/issuance.rs` lines 20–191 and `sighash_versioning.rs` @ 3f9b53fc; the meaning of `[0x00]` is §3.5).

In the `orchard-zsa` object model, the type `IssueBundle<T: IssueAuth>` holds the validating key `ik` (an `IssueValidatingKey<ZSASchnorr>`), a non-empty vector of actions, and an authorization typestate `T`; an `IssueAction` holds the 32-byte `asset_desc_hash`, its `Vec<Note>`, and its `IssuanceFlags`.

3.5 Authorization: BIP-340 over the transaction sighash

The issuance authorization signature scheme `IssueAuthSig` is instantiated as a BIP-340 Schnorr signature over `secp256k1` with the standard parameters, not `RedPallas`; ZIP-227 gives no rationale for that curve choice. Batch verification MAY be used, and precomputation MAY be used if and only if it produces equivalent results (ZIP-227, *Issuance Authorization Signature Scheme*). The associated types (same section):

$$\text{Message} = \mathbb{B}^{\mathbb{Y}^{[N]}}, \quad \text{Signature} = \mathbb{B}^{\mathbb{Y}^{[64]}} \cup \{\perp\}, \quad \text{Public} = \mathbb{B}^{\mathbb{Y}^{[32]}} \cup \{\perp\}, \quad \text{Private} = \mathbb{B}^{\mathbb{Y}^{[32]}},$$

that is, arbitrary byte-sequence messages, 64-byte signatures, 32-byte x-only public keys, and 32-byte secret keys, with $\perp$ as the failure value. Signing fixes the BIP-340 auxiliary data to the all-zero block:

$$a := [0x00]^{32}, \quad \sigma := \text{Sign}(\text{isk}, M) \text{ with auxiliary data } a, \quad \text{issueAuthSig} := [0x00] \parallel \sigma,$$

returning $\perp$ on failure; validation returns 0 if $\sigma = \perp$, else 1 exactly when the BIP-340 `Verify(ik, M,  $\sigma$ )` succeeds² (ZIP-227, *Issuance Authorization Signing and Validation*). With the scheme byte, the two wire encodings are 33 bytes (`ik_encoding`) and 65 bytes (`issueAuthSig`).

²The ZIP’s `Validate` definition writes `Verify(key, M,  $\sigma$ )` for the parameter it names `ik` — an editorial slip we flag when citing (ZIP-227, *Issuance Authorization Signing and Validation*).

What is signed. The message is the transaction SigHash as defined by protocol specification §4.10 under the ZIP-226 modifications, with sighash type SIGHASH_ALL, and the consensus rule requires `IssueAuthSig.Validate(ik, SigHash, issueAuthSig) = 1` (ZIP-227, *Specification: Consensus Rule Changes*). Under ZIP-246 the signature digest includes branch S.5, the issuance digest, identical to the txid branch — so the issuance signature commits to *all* effecting data of the transaction, including the issuance bundle’s own effecting data, though never to any signature (ZIP-246, *Signature Digest*). ZIP-246 also introduces sighash algorithm versioning: each bundle carries `sighashInfo = [sighashVersion] || associatedData`, versions are numbered from 0 per transaction version, version 0 is by convention the “commit to all effecting data” algorithm, and for v6 only version 0 is defined, with empty associated data for the Transparent, Sapling, Orchard, and Issuance bundles — hence the constant `[0x00]` the parser demands (ZIP-246, *Sighash versioning*).

Implementation. The ZSASchnorr scheme wraps the `secp256k1` crate: signing builds a keypair from `isk` and calls `sign_schnorr_with_aux_rand(msg, keypair, &[0u8; 32])` — the ZIP’s $a = [0x00]$ ³² — `ik` is the x-only public key, and `encode()` of key and signature each prepend `ALGORITHM_BYTE = 0x00`, with `decode()` rejecting any other lead byte. Bundle construction is a tpestate chain — `AwaitingNullifier` → `(update_rho)` → `AwaitingSighash` → `(prepare(sighash))` → `Prepared` → `(sign(isk))` → `Signed` — so a bundle cannot be signed before its notes’ ρ values and the sighash exist. On the transaction side, `v6_signature_hash` includes the txid issuance digest, and the builder signs the issue bundle with `prepare(*shielded_sig_commitment)` where that commitment is `signature_hash(tx, SignableInput::Shielded, txid_parts)` — the same SIGHASH_ALL shielded digest the Sapling and Orchard signatures sign. The builder also realizes the $nf_{0,0}$ rule constructively: `first_nullifier` returns the first Action’s nullifier only for an OrchardZSA bundle, and building with an issuance builder but no such bundle fails with `BundleTypeNotSatisfiable`.

A divergence to flag. ZIP-246’s txid and signature digests include a sixth branch, T.6/S.6 `memo_digest` (ZIP-246, *Txid Digest* and *Signature Digest*), but `librustzcash-zsa`’s `to_hash` hashes only the header, transparent, Sapling, Orchard, and issuance branches — the fork does not implement ZIP-231 memo bundles (`transaction/txid.rs` lines 415–460 @ 3f9b53fc). The digest the fork code signs is therefore not byte-identical to the digest ZIP-246 specifies; whichever is corrected, the issuance signature’s coverage claim above should be read against the fork’s five-branch digest today.

3.6 Transaction digests

The v6 digest tree extends the ZIP-244 structure the *Ironwood Guide* presents in §“Transaction digests and signatures”: the txid digest is BLAKE2b-256 with personalization `ZcashTxHash_ || CONSENSUS_BRANCH_ID` over six branches T.1–T.6 (header, transparent, Sapling, Orchard, issuance, memo), and the signature digest has the same six-branch structure with S.5 identical to T.5; a signature digest is produced for each transparent input, Sapling input, OrchardZSA Action, and additionally for each Issuance Action (ZIP-246, *Txid Digest* and *Signature Digest*). The issuance branches are three nested hashes and one authorizing hash, each

over the wire encodings of Table 3 (ZIP-246, *txid_digest* T.5 and A.4: *issuance_auth_digest*):³

```
issuance_digest = BLAKE2b-256(ZTxIdSAIssueHash,
                               issuerLength || issuer || issue_actions_digest),
issue_actions_digest = BLAKE2b-256(ZTxIdIssuActHash,
                                   ||_actions assetDescHash || issue_notes_digest || flagsIssuance),
issue_notes_digest = BLAKE2b-256(ZTxIdIAcNoteHash,
                                   ||_notes recipient || value || rho || rseed),
issuance_auth_digest = BLAKE2b-256(ZTxAuthZSA0rHash, issueAuthSig field encoding),
```

where the A.4 preimage is the ZIP-230 IssueAuthSignature composite

```
sizeSighashInfo || sighashInfo || sizeSignature || signature
```

(ZIP-230, *Issuance Authorization Signature*), and a transaction without issuance components hashes the empty string under the same personalizations.

The code computes exactly these issuance-specific hashes. On the txid side, `hash_issue_bundle_txid_data` hashes

```
compactSize(|ik_encoding|) || ik_encoding || issue_actions_digest
```

under `ZTxIdSAIssueHash`; the actions digest hashes, per action, the 32-byte hash, the notes digest, and the flags byte; the notes digest hashes, per note, the 43-byte recipient, the 8-byte little-endian value, and the two 32-byte fields. On the authorizing side, `hash_issue_bundle_auth_data` hashes

```
compactSize(|sighashInfo|) || sighashInfo || compactSize(65) || [0x00] ||  $\sigma$ ,
```

asserting the 65-byte length and the 0x00 lead. The empty-bundle variants hash the empty string, and all four personalizations match ZIP-246; both digests live in `src/bundle/commitments/issuance.rs` lines 11–86 (orchard-zsa @ cf801a5d). For the wiring into transaction identifiers, `TxidDigester::digest_issue` maps the bundle to its `commitment()`; `to_hash` appends the digest only when the version has `_orchard_zsa()`; and `BlockTxCommitmentDigester` takes the authorizing commitment (`librustzcash-zsa`, `transaction/txid.rs` lines 381–383, 445–452, 595–598 @ 3f9b53fc).

Deterministic regression digests are pinned in the tests — for a seeded two-asset bundle,

```
issuance_digest = ee70e3b61674fd0428ac0020cc4fc5819386e39c4eb3c63357c84c998195bcdb,
issuance_auth_digest = 6df77af7b5323d99376336b770e4c5b06ffc195de81ac7692d1b08b6eb19534d
```

— and cross-implementation test vectors ship in `src/test_vectors/` (`asset_base.rs`, `issuance_auth_sig.rs`), exercised by the asset-base and authorization tests (orchard-zsa, `src/bundle/commitments/issuance.rs` lines 156–190 and `src/test_vectors/` @ cf801a5d). The upstream ZIP-227 Test Vectors section, by contrast, still reads LINK TBD.

3.7 Global issuance state and the consensus rules

Supply integrity is the one property issuance cannot delegate to cryptography: no transaction field simultaneously witnesses everything issued and everything burnt for an Asset, so — along the lines of the ZIP-209 pool turnstile — every node maintains a per-Asset circulation record (ZIP-227, *Rationale for Global Issuance State*).

³Two editorial defects in ZIP-246 to note when citing: the T.5 child list labels `issuerLength/issuer/issue_actions_digest` as T.5a/T.5b/T.5c, while the following section headings call `issue_actions_digest` “T.5a” and `issue_notes_digest` “T.5a.i”; and the A.4 empty case reads “In the case that the transaction has no Orchard Actions” where “no Issuance Actions” is evidently intended.

Definition 3.2 (Global issuance state). The map

$$\begin{aligned} \text{issued_assets} : \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})^* &\rightarrow \{0 \dots \text{MAX_ISSUE}\} \times \{0, 1\} \times \text{Note}^{\text{Issue}}, \\ \text{AssetBase} &\mapsto (\text{balance}, \text{final}, \text{note}_{\text{ref}}), \end{aligned}$$

records, for every issued Asset, the amount in circulation (issued minus burnt), the finalization status — which can never change from 1 to 0 — and the Asset’s reference note. A missing entry ($\text{issued_assets}(\text{AssetBase}) = \perp$) is read as $\text{balance} = 0$, $\text{final} = 0$, $\text{note}_{\text{ref}} = \perp$; the constant $\text{MAX_ISSUE} = 2^{64} - 1$ caps the recorded circulating balance of any Custom Asset at any one state (ZIP-227, *Global Issuance State*).

The map threads through the chain deterministically: it MUST be updated by a node during the processing of any transaction that contains burn information or an issuance bundle; the input state for the OrchardZSA activation block is the empty map; the input state of a block’s first transaction is the final state of the preceding block; each subsequent transaction’s input state is the preceding transaction’s output state; and the block’s final state is its last transaction’s output state (ZIP-227, *Management of the Global Issuance State*).

Per-transaction rules. For a v6 transaction (ZIP-227, *Specification: Consensus Rule Changes*):

- (T1) `nActionGroupsOrchard` MUST be 0 or 1, and `nAGExpiryHeight` MUST be 0; the v6 parser of `librustzcash-zsa` enforces both at deserialization, in `transaction/components/orchard.rs` lines 107–132 @ 3f9b53fc;
- (T2) the output state `issued_assetsOUT` MUST be initialized to the input state `issued_assetsIN`;
- (T3) the `assetBurn` set MUST satisfy the ZIP-226 rules (§4: tuples $(\text{AssetBase}, v)$ with the Native Asset excluded, $0 < v \leq \text{MAX_BURN_VALUE} = 2^{63} - 1$, and no duplicate `AssetBase`; ZIP-226, burn rules); and
- (T4) for every $(\text{AssetBase}, v) \in \text{assetBurn}$, the node MUST check $\text{balance} \geq v$ for the entry of `AssetBase` in `issued_assetsOUT` and subtract v from that balance, *prior to* processing the issuance bundle.

Issuance-bundle rules. For a transaction carrying an issuance bundle (ZIP-227, *Specification: Consensus Rule Changes*):

- (I1) the transaction MUST also contain at least one OrchardZSA Action Group (the ρ /replay rationale of §3.3);
- (I2) the encoding `ik_encoding` MUST be taken from the `issuer` field and MUST start with the byte 0x00 (BIP-340);
- (I3) the signature MUST satisfy `IssueAuthSig.Validate(ik, SigHash, issueAuthSig) = 1`;
- (I4) for each Issue Action: every `IssueNoteDescription` MUST be a valid ZIP-230 encoding; the `AssetBase` is derived from `assetDescHash` and `issuer` (§3.2); and the Asset’s final bit in `issued_assetsOUT` MUST NOT be 1;
- (I5) if $\text{note}_{\text{ref}} = \perp$, the first Issue Note MUST be a reference note (§3.3), and the node MUST set note_{ref} to it;

(I6) for every Issue Note note_j : its ρ MUST equal the value computed per §3.3; the supply update

$$\text{issued_assets}_{\text{OUT}}.\text{balance} + v_j \leq \text{MAX_ISSUE}, \quad \text{issued_assets}_{\text{OUT}}.\text{balance} += v_j$$

MUST hold and be applied; and the node MUST compute the note commitment per ZIP-226;

(I7) if $\text{finalize} = 1$ in flagsIssuance , the node MUST set $\text{final} = 1$.

The commitments enter the tree in a fixed order: for each Action Group of the OrchardZSA bundle, the commitment of every new note per Action; then, for each Issue Action of the issuance bundle, the commitment of every Issue Note (ZIP-227, *Addition to the Note Commitment Tree*).

The rationale ties the pieces together: balances of issued Assets must never go negative, MAX_ISSUE is a practical limit that also lets an issuer create an Asset’s complete supply in a single transaction, and the same map records finalization so that further issuance of a finalized Asset is rejected (ZIP-227, *Rationale for Global Issuance State*). Beyond consensus, the ZIP RECOMMENDS that full validators keep a mutable `issuanceSupplyInfoMap` from `AssetId` to `(totalSupply, finalize)` for wallets and applications tracking total supply (ZIP-227, *Implementing Zcash Nodes*).

The library realization. Function `verify_issue_bundle` takes the bundle, the sighash, a caller-supplied view of global state (`get_global_records`), and the first nullifier. It rejects any sighash kind other than `AllEffecting`, verifies the BIP-340 signature over the 32-byte sighash with the bundle’s `ik`, folds the actions over a `BTreeMap<AssetBase, AssetRecord>`, and returns the updated records — `amount`, `is_finalized`, `reference_note` — for the caller to persist. The fold realizes the per-action and per-note rules as typed errors:

- `IncorrectRhoDerivation` — a note’s ρ differs from the recomputed derivation of §3.3;
- `MissingReferenceNoteOnFirstIssuance` — no record exists for the Asset, and the action’s first note is not a reference note;
- `IssueActionPreviouslyFinalizedAssetBase` — reissuance against a finalized record;
- `IssueBundleIkMismatchAssetBase` and `AssetBaseCannotBeIdentityPoint` — a note’s Asset Base differs from the one `IssueAction::verify` derives from `(ik, assetDescHash)`, or the derived point is the Pallas identity. The latter variant is intended but unreachable at this pin because `AssetBase::custom` panics first (Remark 2.8);
- `ValueOverflow` — the checked `u64` value `sum` overflows, the code’s realization of $\text{MAX_ISSUE} = 2^{64} - 1$.

A signature-skipping variant, `check_issue_bundle_without_sighash`, exists for verifying historical blocks from a trusted checkpoint (`src/issuance.rs` lines 647–705 @ cf801a5d).

Three boundary observations, each load-bearing for anyone re-deriving consensus from these libraries:

- *Empty-action divergence.* The implementation rejects an action with no notes unless it is finalized (`IssueActionWithoutNoteNotFinalized`, `src/issuance.rs` lines 216–219), but ZIP-227’s consensus-rule list contains no such MUST, and ZIP-230 merely notes that `nNotes` may be zero “to only finalize”. For an already-existing Asset, the written consensus rules admit an empty, non-finalizing action that the reference implementation rejects; for a new Asset, the ZIP’s instruction to take the first note as the reference note is undefined when the list is empty.

- *First-issuance detection.* The ZIP keys the reference-note requirement on $\text{note}_{\text{ref}} = \perp$; the code keys it on the complete absence of an `AssetRecord` (`get_global_records` returning `None`). The two are equivalent under the invariant that note_{ref} is set exactly on first issuance — rule (I5) — and no rule ever clears it, so an entry exists if and only if its note_{ref} is set.
- *Node-side enforcement locus.* Neither `orchard-zsa` nor `librustzcash-zsa` contains the `issued_assets` chain-state maintenance or the consensus caller of `verify_issue_bundle`: the libraries return updated records for a node to persist, and the QEDit node integration is not among our local ground-truth repositories. The global-state rules are therefore *specified and library-supported*, with their node enforcement unverified locally.

3.8 Finalization

The `finalize` boolean signals that no further issuance of the specific Custom Asset is possible: every issuance action carries it in `flagsIssuance` (a requirement of the ZIP’s Requirements section), setting it sets the Asset’s final bit — rule (I7) — and transactions attempting to issue further amounts of a previously finalized Asset are rejected — rule (I4) (ZIP-227, *Requirements and Issuance Action*). The bit is monotone: consensus state never changes final from 1 to 0 (Definition 3.2).

Finalization is the ZIP’s whole lifecycle policy, and its Concrete Applications read as patterns over one bit (ZIP-227, *Concrete Applications*):

- *One-time issuance:* set `finalize = 1` on the first action; the entire supply exists from one transaction — `MAX_ISSUE` is sized to permit this (§3.7).
- *Stopping supply:* issue with `finalize = 1` in a final issuance transaction, or in one containing a single note of value 0 (the wire format admits note-less finalizing actions as well, Table 3; the library insists on the finalizing variant, §3.7).
- *NFTs:* issue multiple actions in one transaction, each with `value = 1` at the Asset’s fundamental unit and `finalize = 1` — each action defines a distinct finalized Asset Identifier with supply one, subject to the collision-resistance assumptions of §2.2.

Finalization is also the entirety of the compromise story. No key rotation exists for issuance keys; on compromise of `isk`, the ZIP’s guidance is to set `finalize = 1` for all Assets issued with that key, switch to a new `isk` — hence a new issuer identifier, since `issuer = ik_encoding` — and issue new Assets with new `AssetIds` (ZIP-227, *Issuance Key Compromise*). The old Assets keep circulating and burning; they simply never grow again.

3.9 Fees

The fee treatment of issuance is the sharpest fork/upstream split in the ZSA stack, and we bin it explicitly. The QEDit fork of ZIP-317 adds two contributions to the logical-action count of the conventional fee (the *Wallet Guide*, §“Fees (ZIP-317)”, for the surrounding machinery: $\text{marginal_fee} = 5000 \text{ zatoshis}$, $\text{grace_actions} = 2$):

$$\begin{aligned} \text{contribution}_{\text{ZSAIssuance}} &= n\text{ZSAIssueNotes}, \\ \text{contribution}_{\text{ZSACreation}} &= \text{creation_cost} \cdot n\text{AssetCreations}, \end{aligned}$$

where $n\text{ZSAIssueNotes}$ counts `IssueNote` outputs, $n\text{AssetCreations}$ counts Custom Assets newly added to the global issuance state, and $\text{creation_cost} = 100$ logical actions (fork ZIP-317, *Fee calculation @ zips-zsa fd71419*). The stated rationale: every new Asset adds a row that validators track in perpetuity (§3.7), and the two-orders-of-magnitude creation cost disincentivizes junk assets.

ZIP 317 defines neither contribution. These formulas are *fork-specified*: they describe the experimental ZSA fee policy, not the conventional fee rule for deployed transactions.

4 Transfer and burn (ZIP-226)

ZIP-226 (*Transfer and Burn of Zcash Shielded Assets*; Status: Draft; owners Pablo Kogan and Vivek Arte of QEDit, Daira-Emma Hopwood, and Jack Grigg) defines, jointly with the issuance protocol of ZIP-227 (§3), the OrchardZSA transfer and burn protocol. We develop it as four deltas against the Orchard baseline, each citing the construction it modifies in the *Ironwood Guide* rather than re-deriving it: the note gains one field, the *Asset Base* (§4.1); the note commitment becomes a case split that keeps ZEC commitments identical to NU5 (§4.2); the value commitment trades its fixed value base for the per-asset base, which turns the binding signature into a computational per-asset balance check and gives burn its consensus meaning (§4.3, §4.4); and dummy spends give way to *Split Inputs* for Custom Assets (§5). We then state the extended Action statement (§4.6) and the carrier format with its digests and fees (§4.7).

ZIP 226 assumes ZIP 2005's recoverability changes, whereas the experimental fork does not implement them. Where the difference affects a construction — the split-note ψ^{nf} (§5) and the lead byte (§4.7) — we flag it in place.

4.1 One new field: the OrchardZSA note

Definition 4.1 (OrchardZSA note). An OrchardZSA note extends the Orchard note — the tuple $(\text{addr}, v, \rho, \psi, \text{rcm})$ of the *Ironwood Guide*, “Notes” — by exactly one field:

$$\mathbf{n} := (\text{addr}, v, \text{AssetBase}, \rho, \psi, \text{rcm}),$$

where $\text{AssetBase} \in \mathcal{E}^* := \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{O\}$ is “a valid group element that is not the identity and is not bottom” (ZIP-226; the ZIP writes $\mathbb{P}^*$ for $\mathcal{E}^*$). All other components are as in protocol specification §3.2. Formally,

$$\begin{aligned} \text{Note}^{\text{OrchardZSA}} &:= \{0, 1\}^{88} \times \text{KA}^{\text{Orchard}}.\text{Public} \times \{0, \dots, 2^{64} - 1\} \times \mathcal{E}^* \\ &\quad \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \times \text{NoteCommit}^{\text{Orchard}}.\text{Trapdoor}, \end{aligned}$$

where the ZIP's $\mathbb{F}_{q_p}$ is this series' $\mathbb{F}_{p_{\text{Pallas}}}$ and $\ell_{\text{value}} = 64$ (ZIP-226).

The byte encoding of the new field is the star-encoding of the series' conventions: ZIP-226 sets $\text{asset_base} := \text{LEBS2OSP}_{\ell_p}(\text{repr}_{\mathbb{P}}(\text{AssetBase}))$, which is $\text{AssetBase}^* \in \{0, 1\}^{256}$ in 32 bytes.

In a transfer, an Asset Base is not an arbitrary point chosen by the transactor. Every Custom Asset's base descends from the ZIP-227 issuance chain — constructed in §3, reproduced here because the soundness argument of §5 depends on its shape:

$$\begin{aligned} \text{assetDescHash} &:= \text{BLAKE2b-256}(\text{"ZSA-AssetDescCRH"}, \text{asset_desc}), \\ \text{AssetId} &:= (\text{issuer}, \text{assetDescHash}), \\ \text{EncodeAssetId}(\text{AssetId}) &:= 0x00 \parallel \text{issuer} \parallel \text{assetDescHash}, \\ \text{AssetDigest} &:= \text{BLAKE2b-512}(\text{"ZSA-Asset-Digest"}, \text{EncodeAssetId}(\text{AssetId})), \\ \text{AssetBase} &:= \text{GroupHash}^{\text{Pallas}}(\text{z.cash : OrchardZSA}, \text{AssetDigest}) \end{aligned}$$

(ZIP-227, which names the last map ZSAValueBase). For the native Asset (ZEC/TAZ) the Asset Base is instead *defined* to be

$$V^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash : Orchard-cv}, \text{"v"}),$$

the fixed value-commitment base of the *Ironwood Guide*, “Value commitments” (ZIP-226). This single identification is what keeps every ZEC computation below identical to NU5: ZIP-226 uses $\text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v)$ as $\text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(V^{\text{Orchard}}, v)$. The implementation realizes it as $\text{AssetBase}::\text{zatoshi}() = \text{hash_to_curve}(\text{"z.cash : Orchard-cv"})(\text{b"v"})$.

4.2 The note commitment: a case split

Definition 4.2 (OrchardZSA note commitment). For an OrchardZSA note,

$$\text{NoteCommit}_{\text{rcm}}^{\text{OrchardZSA}}(g_d^*, pk_d^*, v, \rho, \psi, \text{AssetBase}) \\ := \begin{cases} \text{NoteCommit}_{\text{rcm}}^{\text{Orchard}}(g_d^*, pk_d^*, v, \rho, \psi) & \text{if } \text{AssetBase} = V^{\text{Orchard}}, \\ \text{cm}_{\text{ZSA}} & \text{otherwise,} \end{cases}$$

where

$$\text{cm}_{\text{ZSA}} := \text{Sinsemilla}_{\text{z.cash:ZSA-NoteCommit-M}}(g_d^* \parallel pk_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi) \parallel \text{AssetBase}^*) \\ \dot{+} [\text{rcm}] \text{GroupHash}^{\text{Pallas}}(\text{z.cash: Orchard-NoteCommit-r, ""})$$

(ZIP-226; the ZIP writes I2LEBSP for the series' LE_k). The signature is

$$\text{NoteCommit}^{\text{OrchardZSA}} : \text{NoteCommit}^{\text{Orchard}}.\text{Trapdoor} \times \{0, 1\}^{256} \times \{0, 1\}^{256} \times \{0, \dots, 2^{64} - 1\} \\ \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathcal{E}^* \rightarrow \text{NoteCommit}^{\text{Orchard}}.\text{Output},$$

with trapdoor and output types those of $\text{NoteCommit}^{\text{Orchard}}$, unchanged (ZIP-226).

The two branches carry the design's two promises. In the native branch the Asset Base is not hashed at all: a ZEC note commitment under OrchardZSA is the NU5 commitment, bit for bit. In the ZSA branch the message appends AssetBase^* (256 bits) after ψ , against the baseline layout of the *Ironwood Guide*, “Note commitments”:

$$g_d^* \parallel pk_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi) \mapsto \dots \parallel \text{LE}_{255}(\psi) \parallel \text{AssetBase}^*,$$

under the fresh hash domain $\text{z.cash:ZSA-NoteCommit-M}$ but the *same* randomness base $\text{GroupHash}^{\text{Pallas}}(\text{z.cash: Orchard-NoteCommit-r, ""})$. ZIP-226's rationale says that “the nested structure of the Sinsemilla-based commitment allows us to add the Asset Base as a final recursive step”, and calls the output “still indistinguishable from the original Orchard ZEC note commitments”. More precisely, the common output type lets ZSA commitments inhabit the same note commitment tree, while their computational indistinguishability rests on the hiding and hash assumptions of the randomized Sinsemilla commitment (ZIP-226). One caveat bounds the claim: Orchard (v5) note commitments and OrchardZSA (v6) note commitments are contextually distinguishable when observed in transactions, because the two protocols use different transaction versions (ZIP-226).

Remark 4.3 (Implementation). Function `NoteCommitment::derive` computes *both* commitments. For ZEC it calls `CommitDomain::new` on the prefix `"z.cash:Orchard-NoteCommit"`; for ZSA it calls `new_with_separate_domains` on the *M*-prefix `"z.cash:ZSA-NoteCommit"` and the shared *r*-prefix `"z.cash:Orchard-NoteCommit"`. It then selects between them in constant time on `asset.is_zatoshi()`. The sinsemilla crate appends the `-M` and `-r` suffixes to these prefixes.

The nullifier of an ordinary (non-split) OrchardZSA note is generated exactly as in Orchard (protocol specification §4.16; *Ironwood Guide*, “The nullifier”); only Split Inputs use the modified, randomized formula of §5 (ZIP-226).

4.3 Per-asset value: the variable-base value commitment

Definition 4.4 (OrchardZSA net value commitment). Each Action commits to its net value change in its Asset:

$$\begin{aligned} \text{cv}^{\text{net}} &:= \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v^{\text{net}}) = [v^{\text{net}}] \text{AssetBase} + [\text{rcv}] R^{\text{Orchard}}, \\ v^{\text{net}} &:= v_{\text{old}} - v_{\text{new}}, \quad R^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash: Orchard-cv}, "r"), \end{aligned}$$

where AssetBase is the Action’s Asset Base and the randomness base R^{Orchard} is unchanged (ZIP-226).

Exactly one thing changes against the baseline $\text{cv}^{\text{net}} = [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$ of the *Ironwood Guide*, “Value commitments”: the fixed-base multiplication by V^{Orchard} becomes a variable-base multiplication by the Action’s AssetBase, while the randomness term keeps its fixed base (ZIP-226). For ZEC, $\text{AssetBase} = V^{\text{Orchard}}$ and the commitment reduces to the NU5 one (§4.1).

The prover still signs the SIGHASH with $\text{bsk} = \sum_{i=1}^n \text{rcv}_i$ over the n Actions of a transfer (ZIP-226); the verifier’s key acquires burn terms — one zero-trapdoor commitment subtracted per assetBurn entry, each a bare multiple of its base — displayed in this volume as (2) (§5.1). The verifier checks one Pallas-group equation, not independent algebraic coordinates for the assets. Since Pallas is cyclic, every pair of non-identity bases has some discrete-log relation; the separation is computational. After the randomness terms cancel, any nonzero per-asset imbalance that still verifies yields a useful relation among the independently group-hashed Asset Bases, V^{Orchard} , and R^{Orchard} . Under the discrete-log and hash-to-curve assumptions, finding such a relation is infeasible. Subject to those assumptions, the binding signature therefore enforces that each Custom Asset’s net Action value equals its assetBurn entry (or 0 if absent), while the ZEC component equals $v_{\text{balance}}^{\text{Orchard}}$. This is the computational content of ZIP-226’s “per Asset Base” claim; its wording that Pedersen commitments add homomorphically only for the same base is not a literal group-theoretic statement. The binding-signature mechanics — bsk/bvk cancellation and RedPallas on the base R^{Orchard} — are those of the *Ironwood Guide*, “The binding signature”, and we do not restate them.

Remark 4.5 (Implementation: `derive_bvk`). Function `derive_bvk_raw` computes the ZIP equation verbatim: $\sum \text{cv}^{\text{net}}$ minus the zero-trapdoor commitment of `value_balance` at `AssetBase::zatoshi()` minus, over the burn set, the zero-trapdoor commitment of each value at its asset. Function `ValueCommitment::derive` computes $V \cdot \text{value} + R \cdot \text{rcv}$ with V the point `asset.cv_base()` and R the point `hash_to_curve("z.cash:Orchard-cv")` applied to `b"r"`, the value signed per the sign of the value sum.

4.4 Burn

Burning is the value-balance mechanism read as an exit door. The mechanism “does NOT send Assets to a non-spendable address, it simply reduces the total number of units of a given Custom Asset in circulation”; it is enforced at consensus level as an extension of the value-balance mechanism, and it “makes it globally provable that a given amount of a Custom Asset has been destroyed”. The Native Asset (ZEC/TAZ) cannot be burnt this way (ZIP-226).

ZIP-226 attaches three consensus rules to the burn set, whose cardinality it denotes L :

1. for every $(\text{AssetBase}, v) \in \text{assetBurn}$: $\text{AssetBase} \neq V^{\text{Orchard}}$;
2. $v > 0$ and $v \leq \text{MAX_BURN_VALUE} := 2^{63} - 1$;⁴

⁴ZIP-230’s AssetBurn table instead says `valueBurn` is “checked by consensus to be non-zero, and less than `MAX_BURN_VALUE`”. The implementation rejects only amounts strictly greater than `MAX_BURN_VALUE`, i.e. permits equality, agreeing with ZIP-226 (`orchard-zsa, src/bundle/burn_validation.rs`, which defines `MAX_BURN_VALUE = (1 << 63) - 1`). A specification-text defect, not a consensus divergence.

3. every `AssetBase` has at most one entry in `assetBurn`.

Burn data lives *per Action Group*: the `ActionGroupDescription` carries `nAssetBurn` (`compactSize`) and `vAssetBurn`, an array of 40-byte entries — `asset_base` (`byte[32]`) followed by `valueBurn` (`uint64`, little-endian) — so that parties assembling separate Action Groups can each burn their own Assets (withdrawn ZIP-230; §4.7).

The transaction-level field `valueBalanceOrchard` (`int64`) remains the net *ZEC-only* flow, “the net value of Orchard spends minus outputs” (withdrawn ZIP-230), and the transparent protocol is unchanged, so a Custom Asset cannot be unshielded by transfer: “All Custom Assets are contained within the shielded pool” (ZIP-226). Burning is the only supply-reduction path. What a burn reveals on the wire is exactly its `assetBurn` entry: the 32-byte Asset Base and amount — what ZIP-226 calls “the amount and identifier of the Asset being burnt”. In a transfer, “the overall balance is checked without revealing which (or how many distinct) Assets are being transferred”; under the note-encryption and commitment-hiding assumptions, the transferred amounts and Asset Bases remain private (ZIP-226).

A burn also updates ZIP-227’s global issuance state: `issued_assets(AssetBase).balance` $\in \{0, \dots, \text{MAX_ISSUE}\}$ is “computed as the amount of the Asset that has been issued less the amount of the Asset that has been burnt”, and the map **MUST** be updated during processing of any transaction containing burn information (ZIP-227). Supply sufficiency — that a burn not exceed the circulating supply — is therefore a state-level rule of the issuance protocol, not one of ZIP-226’s three per-transaction rules above; we construct the state machine in §3. The implementation enforces both layers side by side: `validate_bundle_burn` returns the per-transaction errors `ZatoshiAsset`, `ZeroAmount`, `InvalidAmount`, and `DuplicateAsset`, and the state-level errors `AssetNotFoundInState` and `InsufficientSupply`.

4.5 Split Inputs

Padding is where the asset dimension bites. An Orchard transfer with more outputs than inputs pads with *dummy* spends: zero-value notes whose Merkle membership check is gated off by v_{old} (constraint A3 of the *Ironwood Guide*, “The Action statement”). For Custom Assets that padding is unsound, because the circuit forces the output note’s Asset Base to equal the input note’s (§4.6) — and a dummy input’s Asset Base is chosen freely by the prover. ZIP-226 states the attack that must be excluded: without enforcing that every Custom Asset input exists in the note commitment tree, “the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds”. The fix: dummy notes remain in use for ZEC notes only; Custom Assets always prove Merkle membership, padding instead with Split Inputs, which force the output’s Asset Base to equal a genuinely issued, GroupHash-derived base (ZIP-226).

The construction is developed once, in §5: the *Split Input* — a copy of a previously issued, tree-resident note of the target Asset, spent with `split_flag` = 1 and its value zeroed in the value-commitment computation only (Definition 5.1); its randomized nullifier with the offset point L^{Orchard} , (3); the three-way divergence over the derivation of ψ^{nf} (§5.6); and the builder mechanics of padding per asset, dummy notes for ZEC and split spends for Custom Assets (§5.8). The three wallet strategies ZIP-226 offers for sourcing the copied note are enumerated in §5.2 (`zip-0226.rst:287–291`). Burn entry points reject duplicate Assets and amounts that do not fit in 63 bits (`add_burn`).

4.6 The Action statement, extended

A bin statement first. ZIP-226’s “Circuit Statement” section specifies the constraint delta in prose and formulas, while deferring “all modifications in the Circuit” to the externally hosted, unversioned document *OrchardZSA circuit*. That document describes the public and private inputs, intermediate values, and constraints, but the ZIP does not embed or pin a complete statement

at the rigor of the *Ironwood Guide*’s Definition “Orchard Action statement”. The pinned implementation is therefore the reproducible source for the exact gate realization, and every gate-level claim below cites it (orchard-zsa, src/circuit/circuit_zsa.rs). Bin: *externally described and implemented, but not fixed by a versioned upstream specification*.

ZIP-226 states the deltas against A1–A9 as follows:

- (a) the Asset Base is witnessed once as an auxiliary input, and both note commitment integrity constraints (A1, A2) switch to $\text{NoteCommit}^{\text{OrchardZSA}}$ with that same witness — one variable simultaneously enforces input/output Asset equality and commitment correctness;
- (b) an auxiliary boolean `is_native_asset` is constrained to equal 1 exactly when $\text{AssetBase} = V^{\text{Orchard}}$;
- (c) `enableZSA = 0` forces `is_native_asset = 1`;
- (d) the value commitment’s fixed-base multiplication (A4) becomes a variable-base multiplication by the witnessed Asset Base;
- (e) a boolean `split_flag` replaces the spent value inside the value commitment: the committed net value becomes $v' - v_{\text{new}}$, with $v' = 0$ if `split_flag = 1` and $v' = v_{\text{old}}$ otherwise; `split_flag = 1` forces `is_native_asset = 0`;
- (f) the Merkle path constraint (A3) becomes $(v_{\text{old}} = 0 \wedge \text{is_native_asset} = 1) \vee \text{ValidMerklePath}$ — the dummy skip survives only for the native Asset;
- (g) nullifier integrity is randomized for split notes (§5).

The circuit realizes these deltas in a single gate, `q_orchard`, whose twelve named constraints are stated once, in §5.5, in the code’s names (the code names the ZIP’s `is_native_asset` as `is_zatoshi_asset`); we do not restate them here.

Two gadgets carry the arithmetic weight. The in-circuit value commitment computes $[\text{magnitude}] \text{asset_base}$ by variable-base long scalar multiplication on the witnessed non-identity point, applies `mul_sign` for the sign, and adds the fixed-base $[\text{rcv}] R^{\text{Orchard}}$ (src/circuit/value_commit_orchard.rs). The in-circuit note commitment shares one Sinsemilla evaluation between the two branches of Definition 4.2: it muxes the initial point Q between Q_{ZEC} (domain `z.cash:Orchard-NoteCommit-M`) and Q_{ZSA} (domain `z.cash:ZSA-NoteCommit-M`; the hard-coded point is test-asserted in src/constants/sinsemilla.rs) on `is_zatoshi_asset`, hashes the common message prefix once, hashes the two suffixes (ZEC: the zero-padded piece h_2 ; ZSA: the Asset Base bits), muxes the resulting hash point, and adds the shared blinding term $[\text{rcm}] R$ — the shared randomness base of Definition 4.2 is what makes this sharing possible (src/circuit/note_commit.rs). The Asset Base message pieces: $h_{\text{ZSA}} = (\text{bits } 249\text{--}253 \text{ of } \psi) \parallel (\text{bit } 254 \text{ of } \psi) \parallel (\text{bits } 0\text{--}3 \text{ of } x(\text{asset}))$; $i = \text{bits } 4\text{--}253 \text{ of } x(\text{asset})$; $j = (\text{bit } 254 \text{ of } x(\text{asset})) \parallel (\tilde{y}(\text{asset})) \parallel 8 \text{ zero bits}$ (ibid.).

The public instance grows from 9 to 10 field elements:

$$(\text{anchor}, \text{cv}^{\text{net}}.x, \text{cv}^{\text{net}}.y, \text{nf}_{\text{old}}, \text{rk}.x, \text{rk}.y, \text{cmx}, \\ \text{enable_spends}, \text{enable_outputs}, \text{enable_zsa}),$$

with the new `enable_zsa` last. In the vanilla flavor `enable_zsa` is always false; the instance-construction API pads the missing tenth element with zero, preserving the vanilla flavor’s public-input semantics. The crate carries two circuit flavors, `OrchardVanilla` and `OrchardZSA` (orchard-zsa, src/circuit.rs, src/flavor.rs). At the bundle level the `enableZSAs` flag is bit 2 of `flagsOrchard` (LSB order, after `enableSpendsOrchard` and `enableOutputsOrchard`; the implementation constant is `FLAG_ZSA_ENABLED = 0b0000_0100`), and coinbase transactions MUST set `enableSpendsOrchard` and `enableZSAs` to 0 (withdrawn ZIP-230).

4.7 The carrier: transaction format, digests, and fees

The following encodings belong to the experimental ZIP 230/246 carrier, not ZIP 229’s deployed v6 format (§1.2). The transfer and burn constructions are draft-specified; their transaction encoding remains *designed but unspecified* for deployment.

Note plaintext. The OrchardZSA note plaintext adds `asset_base` (32 bytes) after `rseed`. Per withdrawn ZIP-230 the v6 Orchard note plaintext is

$$\text{np} := [\text{leadByte}] \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}},$$

with a 32-byte memo key K^{memo} replacing the 512-byte memo field, giving `encCiphertext` of 132 bytes and an OrchardZSAAction of 372 bytes (cv 32 + nullifier 32 + rk 32 + cmx 32 + ephemeralKey 32 + `encCiphertext` 132 + `outCiphertext` 80). ZIP-230 notes that an Orchard note “is essentially equivalent to an OrchardZSA note with `AssetBase` = V^{Orchard} ” and that the two need not be distinguished in plaintexts.

Remark 4.6 (The lead byte is unassigned upstream). Upstream ZIP-226 requires `leadByte` to be the unresolved placeholder `{{ZSALEADBYTE}}` when the protocol’s note-creation sections (§4.7.3 “Sending Notes”, §4.8.3 “Dummy Notes”) are invoked in the computation of ρ and ψ for OrchardZSA notes — a sentence with no counterpart in the fork’s ZIP-226 and no implementation counterpart. Upstream ZIP-230 replaced its original 0x03 with the placeholder `{{LEADBYTE}}` because ZIP-2005 separately claims lead byte 0x03. The QEDit fork ZIPs still say 0x03, and the implementation hard-codes `NOTE_VERSION_BYTE_V3` = 0x03 with `MEMO_SIZE` = 512 and no K^{memo} — the pre-memo-bundle layout, giving `COMPACT_NOTE_SIZE_ZSA` = 52 + 32 = 84 and a 612-byte `encCiphertext`, diverging from ZIP-230’s 132 bytes (orchard-zsa, `src/primitives/zcash_note_encryption_domain.rs`, `src/primitives/orchard_primitives.rs`). No lead byte is currently reserved for ZSA notes upstream. Bin: *open*.

Digests. Withdrawn ZIP-246 extends the ZIP-244 digest tree — the baseline is the *Ironwood Guide*, “Transaction digests and signatures” — with a per-Action-Group `orchard_burn_digest` (node T.4a.vi) under `orchard_action_groups_digest` (personalization `ZTxId0rcActGHash`), beside the compact and non-compact action digests, `flagsOrchard`, `anchorOrchard`, and `nAGExpiryHeight`. The compact digest covers nullifier, cmx, ephemeralKey, and `encCiphertext` (personalization `ZTxId60ActC_Hash`); the non-compact digest covers cv, rk, and `outCiphertext` (`ZTxId60ActN_Hash`). The burn digest is BLAKE2b-256 with personalization `ZTxId0rcBurnHash` over, per `assetBurn` tuple,

$$\text{assetBase} \parallel \text{valueBurn} \quad (64\text{-bit unsigned little-endian}),$$

the empty burn set hashing to `BLAKE2b-256(ZTxId0rcBurnHash, [])` (ZIP-246). One disambiguation: the header-digest field T.1g `burn_amount` is ZIP-233’s network-sustainability burn, unrelated to asset burn (ZIP-246, T.1; ZIP-230, Transaction Format).

Parsing and bundle types. Function `read_v6_bundle` (gated by `zcash_unstable` = “nu7”) reads `nActionGroups`, which must be 0 or exactly 1 (“A V6 transaction must contain at most one action group”), then the actions, flags, anchor, `nAGExpiryHeight` (must be 0), the burn vector (32-byte Asset Base plus note value), the proof, per-action versioned signatures, `valueBalance`, and a versioned binding signature; the write path is symmetric via `write_burn` (`librustzcash-zsa`, `zcash_primitives/src/transaction/components/orchard.rs`). The bundle type carries the burn set as the field `burn: Vec<(AssetBase, NoteValue)>` beside `value_balance`, and `binding_validating_key()` returns `derive_bvk(actions, value_balance, burn)`; the builder collects burns in a `BTreeMap<AssetBase, NoteValue>`, canonically ordered and duplicate-free

by construction (orchard-zsa, src/bundle.rs, src/builder.rs). One open seam: ZIP-230 permits $n\text{ActionGroupsOrchard} > 1$ with per-group burn fields (the multi-party rationale of §4.4), the implementation rejects more than one group, and ZIP-226's bv equation is written over a single flat action/burn set; how multi-group aggregation would interact with the binding signature is unexercised. Bin: *open*.

Tree ordering. Per Action Group, every new OrchardZSA note commitment is appended to the note commitment tree per Action; afterwards, Issue Note commitments are appended per Issue Action (ZIP-227, “Addition to the Note Commitment Tree”; §3).

Fees. The QEDit fork of ZIP-317 adds $\text{contribution_ZSAIssuance}$ and $\text{contribution_ZSACreation}$ terms — issue notes and asset creations respectively — to the conventional-fee formula; it adds no burn term, so a burn contributes to fees only through the Actions that fund it. Upstream ZIP-317 contains no ZSA changes. The conventional-fee baseline is the *Wallet Guide*, “Fees (ZIP-317)”.

5 Split notes and the counterfeiting boundary

An Orchard bundle pairs inputs and outputs into Actions by padding the shorter side with *dummy notes*: an Action with no real input spends a zero-valued note under a freshly sampled key, and the membership constraint waves it through because the check is gated by the spent value — constraint A3 of the Action statement, $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$, which exempts a consumed note of value zero from membership so that an Action with a dummy consumed note can create a real note (the *Ironwood Guide*, “The Action statement”). The construction is protocol §4.8.3, implemented in orchard-zsa, $\text{SpendInfo}::\text{dummy}$: a random spending key, a zero-valued note, and a random Merkle path that is never checked. The exemption is sound there because every Orchard value commitment rides the *fixed* base V^{Orchard} (*Ironwood Guide*, “Value commitments”): a dummy spend can move no value, whatever path it fails to prove. ZIP-226 replaces the fixed base with a witnessed one, and the same exemption becomes a mint. This section develops the attack that forces the change, the *split note* that ZIP-226 substitutes for the dummy note on custom assets, the ZIP-specified circuit delta, and the implemented gate that draws the counterfeiting boundary.

Ground truth for this section: ZIP-226 and ZIP-227 at zcash/zips commit 69610984 (2026-07-09); the pinned implementation, crate orchard of the QED-it fork (orchard-zsa, branch zsa1) at commit cf801a5d (2026-06-29); the QED-it zips fork at fd71419a where it diverges from upstream. Split-note logic lives entirely in the orchard crate; the librustzcash fork consumes its bundle API and contains no split-note code of its own (verified by search at 3f9b53fc).

5.1 The counterfeit construction the boundary excludes

ZIP-226 generalises the net value commitment from the fixed base V^{Orchard} to a per-asset base (ZIP-226, “Value Commitment”):

$$\begin{aligned} \text{cv}^{\text{net}} &:= \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}_{\text{AssetId}}, v_{\text{AssetId}}^{\text{net}}) \\ &= [v_{\text{AssetId}}^{\text{net}}] \text{AssetBase}_{\text{AssetId}} + [\text{rcv}] R^{\text{Orchard}}, \end{aligned} \tag{1}$$

with $v^{\text{net}} := v_{\text{old}} - v_{\text{new}}$ and R^{Orchard} the randomness base of the *Ironwood Guide*’s net value commitment (“Value commitments”), unchanged. The native asset keeps the old base: “the Asset Base of the ZEC Asset will be kept as the original value base point $\mathcal{U}^{\text{Orchard}}$ ” (ZIP-226, “Asset Identifiers”; in code $\text{AssetBase}::\text{zatoshi}()$ is $\text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-cv}, "v")$). Verification subtracts the declared native balance and the declared burns (ZIP-226, “Value Balance

Verification”; implemented in `derive_bvk_raw`):

$$\begin{aligned} \text{bvk} = & \left(\sum_{i=1}^n \text{cv}_i^{\text{net}} \right) - \text{ValueCommit}_0^{\text{OrchardZSA}}(V^{\text{Orchard}}, v^{\text{balanceOrchard}}) \\ & - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} \text{ValueCommit}_0^{\text{OrchardZSA}}(\text{AssetBase}, v), \end{aligned} \quad (2)$$

and the binding signature under bvk proves knowledge of $\text{bsk} = \sum_i \text{rcv}_i$ exactly as in Orchard.

Suppose now that the dummy exemption survived unchanged: a padding Action proves no membership, and — since the base of (1) is a private witness — may witness *any* point A as its asset base. Choose $A := [-1] V^{\text{Orchard}}$, witness $v_{\text{old}} = 0$ and $v_{\text{new}} = v$:

$$\text{cv}_1^{\text{net}} = [0 - v] A + [\text{rcv}_1] R^{\text{Orchard}} = [v] V^{\text{Orchard}} + [\text{rcv}_1] R^{\text{Orchard}}.$$

Pair it with a second Action that outputs a genuine v -valued ZEC note,

$$\text{cv}_2^{\text{net}} = [0 - v] V^{\text{Orchard}} + [\text{rcv}_2] R^{\text{Orchard}},$$

and declare $v^{\text{balanceOrchard}} = 0$ with no burns. The V^{Orchard} -components cancel in (2),

$$\text{bvk} = \text{cv}_1^{\text{net}} + \text{cv}_2^{\text{net}} = [\text{rcv}_1 + \text{rcv}_2] R^{\text{Orchard}},$$

the attacker knows $\text{bsk} = \text{rcv}_1 + \text{rcv}_2$, the binding signature verifies — and a v -valued ZEC note now exists that no one paid for. Suitably chosen known relations among accepted bases can be exploited analogously. (*Classification:* the construction is assembled here from the specified formulas (1) and (2); ZIP-226 states the attack only in words.) The ZIP’s own statement, verbatim: the output note of split Actions “cannot contain just any Asset Base. We must enforce it to be an actual output of a GroupHash computation... Without this enforcement the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds” (ZIP-226, “Split Notes”, rationale).

The boundary to draw is therefore precise: every asset base that enters a value commitment must either be the fixed native base or descend from an accepted GroupHash output. Security then assumes that no useful discrete-log relation among that base, V^{Orchard} , and R^{Orchard} can be found. The one place an unproven base could otherwise enter is an Action with no real input.

5.2 The split input

Definition 5.1 (Split Input, Split Action). ZIP-226 defines a *Split Input* as “an Action input used to ensure that the output note of that Action is of a validly issued AssetBase when there is no corresponding real input note, in situations where the number of outputs are larger than the number of inputs”, and a *Split Action* as an Action containing a Split Input (ZIP-226, “Terminology”).

Mechanically, a Split Input is a *copy of a previously issued note* — one whose commitment is already in the note commitment tree — with exactly two changes: a boolean witness `split_flag` is set to 1, and the note’s value is replaced by 0 *in the value-commitment computation only* (ZIP-226, “Split Notes”). The copied note’s commitment cm_{old} still opens with the true value: in the circuit the value is zeroed solely in the cv^{net} equation, while the old-note commitment check consumes v_{old} unchanged (`orchard-zsa, src/circuit/circuit_zsa.rs`: the gate zeroes v_{old} via the $(1 - \text{split_flag})$ factor in the value equation, lines 127–131, whereas the `NoteCommit` call at lines 637–655 takes v_{old} as witnessed). The Action therefore neither moves value from the copied note nor publishes that note’s ordinary nullifier; instead it publishes the randomized split nullifier of

§5.6. The copy drags the original note’s asset base into the proof, where an equality constraint passes it to the output note.

The split Action participates in the same computational per-asset binding argument as §4.3: a successful imbalance would require a useful discrete-log relation among the independently derived bases. For a correct transaction the custom-asset commitments satisfy

$$\sum_{j \in S_{CA}} cv_j^{\text{net}} - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} [v] \text{AssetBase} = \sum_{j \in S_{CA}} [rcv_j] R^{\text{Orchard}},$$

so the split Action’s $cv^{\text{net}} = [0 - v_{\text{new}}] \text{AssetBase} + [rcv] R^{\text{Orchard}}$ cancels against the real spending Actions of the same asset. The builder implements exactly this: `ActionInfo::value_sum` uses a spent value of 0 when `split_flag` is set, and `derive_bvk_raw` computes (2) over the resulting commitments.

For the note to copy, ZIP-226 offers the wallet three options: (1) another note of the same Asset Base being spent in the same transaction; (2) a different unspent note of that Asset Base, which is *not* thereby spent; (3) an already spent note of that Asset Base — the zeroed value prevents double spending (ZIP-226, “Split Notes”). The ZIP adds that “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before”; §5.7 examines how far that sentence actually holds against the constraint system.

5.3 Membership as well-formedness: the issuance anchor

The fix ZIP-226 specifies, verbatim: “for Custom Assets we enforce that every input note to an OrchardZSA Action must be proven to exist in the set of note commitments in the note commitment tree. We then enforce this real note to be unspendable in the sense that its value will be zeroed in split Actions and the nullifier will be randomized... the proof itself ensures that the output note is of the same Asset Base as the input note. In the circuit, the split note functionality will be activated by a boolean private input (aka the `split_flag` boolean)” (ZIP-226, “Split Notes”, rationale).

Why does tree membership prove that an asset base is a GroupHash output? Because for custom assets the tree is fed exclusively from two sources that both enforce it. Transfer outputs inherit their base from a proven input by the in-circuit equality constraint (§5.4). Issuance outputs are checked publicly: a consensus rule obliges validators to recompute each Issue Note’s AssetBase from the issuance bundle’s issuer key and the IssueAction’s `assetDescHash` (ZIP-227, “Specification: Consensus Rule Changes”), along the derivation chain constructed in §2.3 (ZIP-227, “Asset Bases”), and then appends the issue-note commitments to the *same* Orchard note commitment tree — transfer-Action commitments first, then issuance notes (ZIP-227, “Addition to the Note Commitment Tree”). Hence, under note-commitment binding, a custom-asset note commitment resident in the tree commits, by induction along the two rules, to a genuine GroupHash output. ZIP-226 states the conclusion: “This ensures that the value base points of all output notes of a transfer are actual outputs of a GroupHash, as they originate in the Issuance protocol which is publicly verified” (“Split Notes”, rationale); and, on the identifier side, “We prevent a potential malleability attack on the Asset Identifier by ensuring the output notes receive an Asset Base that exists on the global state” (“Rationale for Asset Identifiers”).

One clarification the ZIP’s sentence invites: “every input note to an OrchardZSA Action” is scoped to *custom-asset* inputs. Native-asset (ZEC) Actions in a v6 bundle retain the dummy-note exemption unchanged — “The ZEC-based Actions will still include dummy input notes, whereas the OrchardZSA Actions will include split input notes and will not include dummy input notes” (ZIP-226, “Backward Compatibility”; the “Split Notes” rationale states the same). The constraint-level form of this scoping is the disjunction in §5.4.

5.4 The specified circuit delta

We state ZIP-226’s circuit changes (“Circuit Statement”) as a delta against the Action statement A1–A9 of the *Ironwood Guide* (“The Action statement”). Two new boolean witnesses appear: `split_flag`, and `is_native_asset`, constrained to equal 1 exactly when the witnessed base is the native one, $\text{AssetBase} = V^{\text{Orchard}}$. One public input appears: the bundle-level `enableZSA` flag (a bit of the `v6 flagsOrchard` field), with the constraint $\text{enableZSA} = 0 \implies \text{is_native_asset} = 1$ — switching the flag off collapses the circuit to native-asset behaviour and, transitively via the split constraint below, disables split notes (ZIP-226, “Overview” and “Circuit Statement”, The `enableZSA` Flag; in code the flag is the tenth public input, taken from `Flags::zsa_enabled()`, `orchard-zsa`, `src/circuit.rs`, lines 476–494, 560).

- **A1, A2 (note commitment integrity).** Both commitment checks replace $\text{NoteCommit}^{\text{Orchard}}$ with $\text{NoteCommit}^{\text{OrchardZSA}}$, which takes the witnessed `AssetBase` as an additional final argument; the *same* once-witnessed base feeds the old-note check, the new-note check, and the value commitment (ZIP-226, “Circuit Statement”, Asset Base Equality). For the native asset $\text{NoteCommit}^{\text{OrchardZSA}}$ coincides with $\text{NoteCommit}^{\text{Orchard}}$; otherwise it is the Sinsemilla commitment over the extended message ending in the asset-base encoding, in the hash domain “z.cash:ZSA-NoteCommit-M” with the blinding base shared with Orchard (“z.cash:Orchard-NoteCommit-r”) (ZIP-226, “Note Structure and Commitment”). The full $\text{NoteCommit}^{\text{OrchardZSA}}$ construction is Definition 4.2; its in-circuit mux is described in §4.6.
- **A3 (membership).** The value gate becomes a disjunction (ZIP-226, “Circuit Statement”, Asset Identifier Consistency for Split Actions — Merkle Path Validity):

$$(v_{\text{old}} = 0 \wedge \text{is_native_asset} = 1) \vee (\text{Valid Merkle Path}),$$

so the dummy skip survives *only* for the native asset. Every custom-asset input — split or not, zero-valued or not — must prove membership.

- **A4 (value commitment).** Two changes. First, “the fixed-base multiplication constraint between the value and the value base point of the value commitment, `cv`, is replaced with a variable-base multiplication between the two” (ZIP-226, “Circuit Statement”, Value Commitment Correctness), the base being the witnessed `AssetBase`. Second, the input value is replaced by a generic v' (ZIP-226, “Circuit Statement”, Asset Identifier Consistency for Split Actions):

$$\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{OrchardZSA}}(\text{AssetBase}, v' - v_{\text{new}}), \quad v' = \begin{cases} 0 & \text{if } \text{split_flag} = 1, \\ v_{\text{old}} & \text{otherwise,} \end{cases}$$

together with $\text{split_flag} = 1 \implies \text{is_native_asset} = 0$: split notes exist only for custom assets, and a zero-valued native dummy cannot masquerade as one.

- **A5 (nullifier).** The derivation changes for split notes “to prevent the identification of notes” (ZIP-226, “Circuit Statement”, Asset Identifier Consistency for Split Actions); §5.6 develops it.
- **A6, A7 (spend authority, address integrity).** Untouched by the ZIP’s change list — and, decisively for §5.7, they remain *unconditional*: no `split_flag` factor relaxes them.

5.5 The implemented gate

A naming note, once: the ZIP’s `is_native_asset` is `is_zatoshi_asset` throughout the implementation, assigned 1 exactly when the witnessed asset equals `AssetBase::zatoshi()` (`orchard-zsa`, `src/circuit/gadget.rs`, `circuit_zsa.rs`). We keep the ZIP name in prose and the code name inside code-derived formulas.

The implemented circuit concentrates the delta in a single gate, `q_orchard`, which carries twelve named constraints where the vanilla circuit’s corresponding gate has four (`src/circuit/circuit_zsa.rs`, lines 68–185, versus `src/circuit/circuit_vanilla.rs`, lines 59–100). In the code’s names:

```
bool_check(split_flag),    bool_check(is_zatoshi_asset),
v_old · (1 – split_flag) – v_new = magnitude · sign,
(v_old + 1 – is_zatoshi_asset) · (root – anchor) = 0,
v_old = 0 ∨ enable_spends = 1,    v_new = 0 ∨ enable_outputs = 1,
is_zatoshi_asset · (asset_x – zatoshi_x) = 0,    is_zatoshi_asset · (asset_y – zatoshi_y) = 0,
(1 – is_zatoshi_asset) · (Δx · Δxinv – 1) · (Δy · Δyinv – 1) = 0,
(1 – split_flag) · (ψold – ψnf) = 0,    split_flag · is_zatoshi_asset = 0,
(1 – enable_zsa) · (1 – is_zatoshi_asset) = 0,
```

where Δ_x, Δ_y abbreviate the code’s `diff_asset_x, diff_asset_y` — the witnessed asset’s coordinates minus the native base’s — and ψ^{nf} is the code’s `psi_nf` (§5.6). Four points merit unpacking.

The folded value equation. The single constraint $v_{old} \cdot (1 - \text{split_flag}) - v_{new} = \text{magnitude} \cdot \text{sign}$ folds the ZIP’s case-split v' into one expression; the witness generator supplies the matching net value, $-v_{new}$ for split spends and $v_{old} - v_{new}$ otherwise (`circuit_zsa.rs`, lines 462–501).

The Merkle factor. The disjunction of A3 is encoded as the single product $(v_{old} + 1 - \text{is_zatoshi_asset}) \cdot (\text{root} - \text{anchor}) = 0$. The code comment argues the encoding safe from wrap-around: `is_zatoshi_asset` is boolean-checked and v_{old} is range-checked to 64 bits in the note-commitment evaluation, so over $\mathbb{F}_{p_{\text{Pallas}}}$ the first factor vanishes exactly when $v_{old} = 0$ and `is_zatoshi_asset` = 1 (`circuit_zsa.rs`, lines 132–139). The consequence bears repeating: for every custom-asset input the factor is nonzero, so `root` = `anchor` is mandatory — a strictly stronger membership requirement than vanilla Orchard’s v_{old} -gate, applying even to zero-valued non-split custom-asset inputs.

Proving inequality. The assignment `is_zatoshi_asset` = 0 must itself be sound: the circuit must verify that the witnessed base *differs* from V^{Orchard} . The prover witnesses field inverses $\Delta_x^{\text{inv}}, \Delta_y^{\text{inv}}$ (assigned 0 when the corresponding difference vanishes), and the triple product above forces at least one difference to be exhibited invertible (`circuit_zsa.rs`, lines 160–170 for the gate, 792–841 for the assignment).

The variable-base value commitment. With the base witnessed, the vanilla circuit’s fixed-base *short* multiplication $[v^{\text{net}}] V^{\text{Orchard}}$ — whose 64-bit range check is implicit in the short-multiplication decomposition — is no longer available. The ZSA gadget makes the range check explicit: the magnitude is decomposed against the Sinsemilla lookup table as six 10-bit windows plus one 4-bit short check (64 bits), then promoted to a full-width scalar (`ScalarVar::from_base`) and fed to the variable-base long multiplication — the code notes it is “optimized for ~255-bit scalar” and leaves a TODO to specialise it for 64 bits — after which the sign is applied and the blind $[rcv] R^{\text{Orchard}}$ added (`src/circuit/value_commit_orchard.rs`, lines 38–127). Two supporting details: the ZSA circuit uses the extended lookup configuration `PallasLookupRangeCheck4_5BConfig` (an additional lookup-table column for 4- and 5-bit short checks), and it refuses to synthesise under `CircuitVersion::InsecureUnanchoredBase`, the pre-NU6.2 `halo2_gadgets` variable-base scalar-multiplication version whose base point was not anchored — directly load-bearing here, because the entire counterfeiting boundary rests on the

multiplication being by the *witnessed* AssetBase and nothing else (`circuit_zsa.rs`, lines 22, 51, 190–197, 334–337; `circuit.rs`, lines 151–185).

The flag interactions are tested asymmetrically: the three legal combinations of

$$(\text{is_zatoshi_asset}, \text{split_flag})$$

— (1,0), (0,1), and (0,0) — are proven passing, while the (1,1) combination is excluded at witness-generation time by the test helper’s assertion (“We cannot create a split note with a zatoshi asset”), so the constraint $\text{split_flag} \cdot \text{is_zatoshi_asset} = 0$ is not itself exercised by a failing-proof test (`circuit_zsa.rs`, lines 1174–1312). Documented cost: the ZSA proof measures 5120 bytes for one Action and 7392 for two, against the vanilla 4992 and 7264, at the shared circuit size $K = 11$; the `enableZSA` bit is the tenth public input, at index 9 (the proof-size tests of `circuit_zsa.rs` and `circuit_vanilla.rs`; `circuit.rs`, lines 68–80, 536–563).

5.6 The split nullifier

A split input copies a note that may sit unspent in someone’s wallet; its Action must nevertheless publish *some* nullifier. Publishing the note’s true nullifier would be harmful in both directions: it would mark an unspent note spent (bricking it), and it would collide with the honest nullifier if the same note were spent for real in the same transaction. ZIP-226 therefore randomises the split note’s nullifier (“Split Notes”):

$$\text{nf}_{\text{old}} = \text{Extract}([F_{\text{nk}}(\rho_{\text{old}}) + \psi^{\text{nf}}] G^{\text{nf}} + \text{cm}_{\text{old}} + L^{\text{Orchard}}), \quad (3)$$

where F_{nk} and G^{nf} abbreviate the nullifier PRF $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$ and the nullifier base K^{Orchard} of condition A5 of the *Ironwood Guide*’s Definition “Orchard Action statement” (ZIP-226 writes $\mathcal{K}^{\text{Orchard}}$ for G^{nf} and computes the bracketed sum in the Pallas base field). Two deltas against A5: the note’s ψ_{old} is replaced by a fresh ψ^{nf} , and a fixed offset point

$$L^{\text{Orchard}} := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, "L")$$

is added (ZIP-226, “Split Notes”; the constant is pinned and test-asserted against the `GroupHash` evaluation in `orchard-zsa`, `src/constants/nullifier_1.rs`). The gate constraint $(1 - \text{split_flag}) \cdot (\psi_{\text{old}} - \psi^{\text{nf}}) = 0$ of §5.5 makes the substitution split-only: non-split OrchardZSA nullifiers coincide with the Orchard derivation, as the ZIP states (“Note Structure and Commitment”: the non-split nullifier “is generated in the same manner as in the Orchard protocol”).

In circuit, the nullifier gadget computes the standard point $\text{cm}_{\text{old}} + [F_{\text{nk}}(\rho_{\text{old}}) + \psi^{\text{nf}}] G^{\text{nf}}$ from the witnessed ψ^{nf} , forms the offset variant by adding the constant L^{Orchard} , and muxes the two on `split_flag` before extraction (`src/circuit/derive_nullifier.rs`, lines 54–111). Out of circuit, `Nullifier::derive` mirrors the computation and selects the offset variant in constant time exactly when the note carries a `rseed_split_note` (`src/note/nullifier.rs`, lines 73–90; `src/note.rs`, lines 415–426).

Deriving ψ^{nf} : a three-way divergence. The value of ψ^{nf} is always constrained through the A5 equation to the published nullifier. Only the equality $\psi^{\text{nf}} = \psi_{\text{old}}$ is gated off in the split case; no constraint derives ψ^{nf} from a seed. How a wallet samples it for a split note is therefore wallet-level, not consensus-level. The three extant texts disagree; per the volume’s policy (§1.3), the upstream ZIP formula comes first. Upstream ZIP-226 — written against the ZIP-2005 baseline, “expected to deploy before any ZSA activation” — specifies

$$\psi^{\text{nf}} := \text{ToBase}(\text{PRFexpand}_{\text{rseed_nf}}(0x0A \parallel \rho_{\text{old}}))$$

with `rseed_nf` uniform on 32 bytes and the domain byte `0x0A` “reserved by ZIP 2005 for this use” (`zip-0226.rst`:297 @ 69610984). The QED-it zips fork specifies neither derivation: there ψ^{nf}

“is sampled uniformly at random” on the Pallas base field (fork zip-0226.rst:293 @ fd71419a). The implementation derives

$$\psi^{\text{nf}} := \text{ToBase}(\text{PRFexpand}_{\text{rseed_split_note}}(0x09 \parallel \rho_{\text{old}})),$$

the ordinary ψ domain byte keyed by a fresh random 32-byte `rseed_split_note` (orchard-zsa, src/circuit.rs:316–319, src/note.rs:114–116 and 417–425; domain byte PSI = 0x09 per the QED-it zcash_spec fork, src/prf_expand.rs:89 @ d5e84264). Direct field sampling is uniform; the two PRF-and-reduction constructions are intended to be pseudorandom and have at most the negligible modular-reduction bias of `ToBase`. The implementation matches neither ZIP text exactly. (*Classification*: each derivation is specified in its own text; the divergence is consensus-invisible.)

Why the offset L^{Orchard} ? The randomised ψ^{nf} is intended to make the split nullifier unlinkable to the note’s true nullifier. ZIP-226 states no rationale for additionally adding L^{Orchard} . A plausible purpose is robustness against a prover who *chooses* $\psi^{\text{nf}} = \psi_{\text{old}}$ — the circuit does not forbid it in the split case — and would thereby publish the note’s true nullifier from a zero-valued split Action, bricking the original note or colliding with an honest spend of it elsewhere in the transaction. The constant offset makes the pre-extraction group point different even under that choice. It is not an absolute noncollision proof for the extracted nullifier, since elliptic-curve points with opposite signs have the same extracted x -coordinate. (*Classification*: designed-but-unspecified; the preceding sentence is our inference, stated in neither ZIP nor code.)

5.7 Ownership, in fact and in prose

Recall the ZIP’s claim that “we do not care about whether the previously issued note copied to create a Split Input is owned by the sender” (§5.2). Both the proof and the Action signature narrow that claim. Spend-authority consistency (A6) and address integrity (A7) are unchanged and unconditional in the ZSA circuit: the proof establishes $\text{rk} = [\alpha] G^{\text{SpendAuth}} + \text{ak}$ and $\text{ivk} = \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk})$ with $\text{pk}_{\text{dold}} = [\text{ivk}] \text{g}_{\text{dold}}$ — for split inputs too (orchard-zsa, src/circuit/circuit_zsa.rs, lines 554–623; the old-note commitment equality at lines 625–659). Because the copied note’s cm_{old} must open to the actual tree leaf (Sinsemilla binding plus the mandatory Merkle check of §5.5), the witnessed pk_{dold} is the original recipient’s, and A7 then forces the prover to know $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ consistent with it — the full-viewing-key components or an equivalent witness. More decisively, every Action must carry a RedPallas spend authorization signature under rk . A split spend has `dummy_sk = None`; preparation auto-signs only true dummy spends, and `apply_signatures` finishes a split Action only after receiving a `SpendAuthorizingKey` whose validating key matches ak , otherwise returning `MissingSignatures` (src/builder.rs, lines 308–315 and 1347–1432, `SpendInfo::create_split_spend`, `Bundle::prepare`, `Bundle::apply_signatures`, and `Bundle::sign`). Thus constructing a valid transaction requires the copied note’s spend-authorizing key, absent a signature forgery or discrete-log break. The ZIP’s sentence holds for *spentness* — a nullified note can still be copied because its value is zeroed — but not for ownership. The builder consequently copies the first real spend of that Asset in the same bundle, option (1) of §5.2 (src/builder.rs, `pad_spend`). (*Classification*: the circuit and signature behavior are implemented; the ZIP rationale overstates the available choices.)

One genuine exception exists, and neither ZIP states it. ZIP-227 requires the first note of any asset’s first issuance to be a *reference note*: value 0, recipient the default diversified address of the all-zero Orchard spending key — a key anyone can derive (ZIP-227, “Reference Notes”; orchard-zsa, src/constants/reference_keys.rs pins the all-zero spending key and the 43-byte raw address). Because that spending key is public, anyone can satisfy A6/A7 and produce the Action signature for the reference note, which makes it a universally available split input

for its asset — the one case in which “ownership does not matter” holds literally. (*Classification: inference*; the connection between reference notes and split inputs is stated in neither ZIP nor the implementation.)

5.8 Builder mechanics: padding, per asset

The two padding regimes coexist per ZIP-226’s rule quoted in §5.3: dummy inputs for ZEC Actions, split inputs for custom-asset Actions. The builder partitions spends and outputs by asset, pads each side of each partition to equal length, shuffles within each asset, then shuffles the assembled Actions across assets (orchard-zsa, src/builder.rs, lines 1081–1164). The padding spend is asset-dependent (pad_spend, lines 928–944): for the native asset, SpendInfo::dummy exactly as in Orchard; for a custom asset, create_split_spend on the first real spend of that asset in the bundle. If the bundle spends none, pad_spend produces BuildError::NoSplitNoteAvailable; the current per-asset iterator unwraps that result and panics with the error rather than returning it from bundle_for_version. Unlike a dummy, a split input cannot be conjured from nothing. Output-side padding uses dummy outputs of the padding asset in both regimes.

The copy itself is minimal. Function Note::create_split_note duplicates the note and sets exactly one new field, rseed_split_note — a fresh random 32-byte seed, drawn by RandomSeed::random and resampled until the derived esk is valid — asserting the asset is not the native one; SpendInfo::create_split_spend keeps the original note’s full viewing key and Merkle path, sets no dummy signing key, and raises split_flag (src/note.rs, lines 436–442, 95–103; src/builder.rs, lines 302–318). Recipient, value, asset, ρ , rseed — hence cm_{old} — are identical to the tree-resident original; the only witness-level differences downstream are split_flag = 1 and the ψ^{nf} drawn from the new seed. Witness assembly routes exactly that: Witnesses::from_action_context_unchecked takes ψ^{nf} from rseed_split_note when present and from rseed otherwise — so $\psi^{nf} = \psi_{old}$ for every non-split spend — and packs (ψ^{nf} , asset, split_flag) into the ZSA witness extension that the vanilla flavour leaves unknown (src/circuit.rs, lines 199–244, 300–345).

The counterfeiting boundary, restated once in full: a custom-asset output note’s base equals its Action’s input-note base (the A1/A2 equality on the shared witness); that input note is a tree leaf (the mandatory Merkle factor); under note-commitment binding, every custom-asset tree leaf descends from a publicly recomputed GroupHash output (the issuance anchor of §5.3); and the copy that makes padding possible moves no value (the zeroed v'). Those links enforce the counterfeiting boundary of §5.1. Independently, the randomized nullifier (3) avoids publishing the original note’s ordinary nullifier except with negligible collision probability under honest randomisation; it is not an absolute noncollision guarantee (§5.6).

6 Transaction integration and outlook

The preceding sections construct the OrchardZSA objects — asset bases (§2), notes and Actions (§4, §5), issuance (§3), and burn (§4.4). This section places them in a transaction: the v6 wire format (ZIP-230), the digest tree that yields transaction identifiers, sighashes, and the authorizing-data commitment (ZIP-246), the transaction-level consensus rules of ZIP-226/227, the fee treatment (ZIP-317), the state of the QED-it implementation stack at the pins in Table 1, and the obligations the format imposes on wallets. The format is experimental; §1.2 states its deployment boundary.

6.1 Experimental and deployed carriers

The carrier below is the experimental OrchardZSA format of ZIP 230, with digests from ZIP 246. Both ZIPs are withdrawn. Its version number 6 must not be confused with ZIP 229’s deployed

version 6 format, which carries Ironwood and no ZSA bundles. The two byte formats are incompatible despite the shared number.

We use the experimental format to explain how the note, issuance, burn, and signature constructions compose into one transaction. Its fields and digests are *designed but unspecified* for deployment; the remaining specification obligations are in §6.8.

6.2 The experimental wire format (ZIP-230)

Common and transparent fields. The common header carries `header` (`fOverwintered` and the version), `nVersionGroupId`, `nConsensusBranchId`, `lock_time`, `nExpiryHeight`, an explicit 8-byte fee (`uint64`, per ZIP-2002), and an 8-byte `zip233Amount` (`uint64`, per ZIP-233) (ZIP-230, Transaction Format). Making the fee an explicit field replaces its derivation as the residual difference between transaction input and output value and lets clients read it without resolving prior transparent outputs (ZIP-2002). Separately, signatures become versioned below. The transparent fields add `vSighashInfo`: one `TransparentSighashInfo` (a `compactSize` length followed by `sighashInfo` bytes) per transparent input (ZIP-230, Transaction Format).

Orchard fields: Action Groups. The Orchard component becomes a vector of *Action Groups*: `nActionGroupsOrchard`, `vActionGroupsOrchard`, then a transaction-level `valueBalanceOrchard` (`int64`) and `bindingSigOrchard`. The balance and binding signature are present iff `nActionGroupsOrchard > 0`; an absent `valueBalanceOrchard` means $v_{\text{balance}}^{\text{Orchard}} = 0$ (ZIP-230, Transaction Format). Each `ActionGroupDescription` contains, in order (ZIP-230, OrchardZSA Action Group Description):

- `nActionsOrchard` (`compactSize`, MUST be > 0) and `vActionsOrchard`, at 372 bytes per OrchardZSAAction;
- `flagsOrchard`, LSB-first: `enableSpendsOrchard`, `enableOutputsOrchard`, `enableZSAs`, remaining bits zero;
- `anchorOrchard` (32 bytes) — each group carries its own anchor;
- `nAGExpiryHeight` (`uint32`);
- `nAssetBurn` and `vAssetBurn`, at 40 bytes per `AssetBurn` entry;
- `sizeProofsOrchard` and `proofsOrchard`, one aggregated proof for the group;
- `vSpendAuthSigsOrchard`, one `OrchardSignature` per Action.

Field `nAGExpiryHeight` exists for forward compatibility with ZSA atomic swaps. ZIP-228 is now a Draft that uses this field to expire swap orders, but it still depends on the withdrawn ZIP-230/246 carrier; for the core OrchardZSA proposal the field is zero by consensus (ZIP-230, Rationale for `nAGExpiryHeight`; §6.4). This restriction concerns `nAGExpiryHeight`, not the transaction-level expiry field `nExpiryHeight`. Burn fields sit inside each Action Group rather than at transaction level so that, in a future multi-party Action Group world, each party can burn assets within its own group (ZIP-230, Rationale for including Burn fields inside OrchardZSA Action Groups).

Action and burn encodings. An `OrchardZSAAction` occupies 372 bytes: `cv` (32), `nullifier` (32), `rk` (32), `cmx` (32), `ephemeralKey` (32), `encCiphertext` (132), and `outCiphertext` (80) (ZIP-230, OrchardZSA Action Group Description). The 132-byte ciphertext is the memo-bundle-era layout: the note plaintext gains the 32-byte asset base and replaces the in-band 512-byte memo with a 32-byte memo key K^{memo} (ZIP-231), the memo chunks themselves moving to a transaction-level memo bundle (fields `nMemoChunks`, `pruned`, `vMemoChunks`, present iff `fAllPruned = 0`)

whose pruned entries are replaced by their digests (ZIP-230, Transaction Format; §6.3). The v6 note plaintext is

$$\text{leadByte} \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{asset_base} \parallel K^{\text{memo}},$$

with $\text{asset_base} = \text{LEBS2OSP}_{\ell_p}(\text{repr}_p(\text{AssetBase}))$, and the lead byte mandatory for all v6 note plaintexts (ZIP-230, note plaintext encoding). Upstream, the lead byte is the unassigned placeholder `{LEADBYTE}`: the original assignment 0x03 was taken by ZIP-2005, and no replacement is assigned (§6.6). For the same K^{memo} reason the Sapling `OutputDescriptionV6` shrinks `encCiphertext` to 100 bytes (ZIP-230, Sapling Output Description). An `AssetBurn` entry is 40 bytes: `AssetBase` (32, the encoding of `AssetBaseOrchard`) and `valueBurn` (`uint64`). ZIP-230 says the amount is non-zero and strictly less than `MAX_BURN_VALUE`, but ZIP-226’s consensus rule and the pinned code permit equality (§4.4).

Issuance bundle. The issuance component carries five fields: `issuerLength` (`compactSize`), `issuer`, `nIssueActions`, `vIssueActions`, and `issueAuthSig`. The signature is present iff `nIssueActions > 0`; the first four fields are always present, and if `nIssueActions = 0` then `issuerLength` MUST be 0 — an empty issuance bundle is exactly two zero `compactSize` values (ZIP-230, Transaction Format). An `IssueAction` carries `assetDescHash` (32 bytes), `nNotes` and `vNotes` at 115 bytes per `IssueNoteDescription`, and `flagsIssuance` with `finalize` in the LSB; `nNotes` may be zero, so a finalize-only action for an existing Asset is expressible; the new-Asset corner case remains undefined in ZIP 227 (§3.7). An `IssueNoteDescription` is recipient (43 bytes, the raw Orchard payment-address encoding), `value` (`uint64`), `rho` (32), `rseed` (32). The `IssueAuthSignature` is `sizeSighashInfo` and `sighashInfo`, then `sizeSignature` and a signature “preceded by a 0x00 byte indicating a BIP 340 signature” (ZIP-230, issuance field encodings) — the same 0x00 scheme byte that prefixes the issuer key encoding (ZIP-227, Issuance Authorization Signature Scheme).

Versioned signatures. The `SaplingSignature` and `OrchardSignature` types prepend a `compactSize`-prefixed `sighashInfo` to the 64-byte RedJubjub or RedPallas signature (ZIP-230, signature encodings). ZIP-246 defines the versioning:

$$\text{sighashInfo} := [\text{sighashVersion}] \parallel \text{associatedData},$$

where version 0 is the “commit to all effecting data” algorithm; for v6, only version 0 is defined, with empty `associatedData`, for all four of Transparent, Sapling, Orchard, and Issuance (ZIP-246, Sighash versioning). The machinery buys the ability to introduce — and, critically, to retire — sighash algorithms per transaction version without a new format.

Coinbase. For coinbase transactions the `enableSpendsOrchard` and `enableZSAs` bits MUST be zero (ZIP-230, consensus rules).

6.3 Digests: txid, sighash, and authorizing data (ZIP-246)

ZIP-246 extends the ZIP-244 digest tree — constructed for vanilla Orchard in the *Ironwood Guide*, §“Transaction digests and signatures” — from four top-level branches to six:

$$\text{txid} := \text{BLAKE2b-256}(\text{ZcashTxHash}_\parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{transp}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}} \parallel d_{\text{issue}} \parallel d_{\text{memo}}),$$

the nodes T.1–T.6 of the ZIP, each itself a personalized BLAKE2b-256 (ZIP-246, TxId Digest). New relative to ZIP-244 are the fee and burn fields in the header digest, the Sapling output digests, the

entire Orchard Action-Group subtree, the issuance subtree, and the memo subtree. The header digest T.1 is

$$d_{\text{hdr}} := \text{BLAKE2b-256}(\text{ZTxIdHeadersHash}, \text{version} \parallel \text{versionGroupId} \parallel \text{branchid} \parallel \text{lockTime} \parallel \text{expiryHeight} \parallel \text{LE}_{64}(\text{fee}) \parallel \text{LE}_{64}(\text{burnAmount}))$$

(ZIP-246, T.1). The Orchard branch T.4 hashes the concatenated per-group digests and the balance, $d_{\text{Orch}} = \text{BLAKE2b-256}(\text{ZTxIdOrchardHash}, d_{\text{AGs}} \parallel \text{LE}_{64}(v_{\text{balance}}^{\text{Orchard}}))$, with a defined empty case; each group digest (T.4a) covers a compact digest (nf, cmx, ephemeralKey, encCiphertext), a non-compact digest (cv, rk, outCiphertext), flagsOrchard, anchorOrchard, nAGExpiryHeight, and a burn digest over the (AssetBase, valueBurn) tuples (ZIP-246, T.4a). The issuance branch T.5 hashes issuerLength || issuer and a per-action digest (per action: assetDescHash, a per-note digest over (recipient, value, ρ , rseed), flagsIssuance), each level with a defined empty case (ZIP-246, T.5). The memo branch T.6 hashes a nonce and the concatenation of per-chunk digests; the v6 pruned memo bundle stores exactly these memo_chunk_digest / memo_digest values in place of pruned chunks, so pruning preserves the txid (ZIP-246, T.6; ZIP-230, Transaction Format). Table 4 collects the personalization strings; the implementation defines the same constants (§6.6).

Node	Personalization	Content
T.1	ZTxIdHeadersHash	header, incl. fee and burn amount
T.3b.i	ZTxId6SOutC_Hash	Sapling v6 outputs, compact
T.3b.ii	ZTxId6SOutN_Hash	Sapling v6 outputs, non-compact
T.4	ZTxIdOrchardHash	Action-Group digests, balance
T.4a	ZTxIdOrcActGHash	one Action Group
T.4a.i	ZTxId60ActC_Hash	nf, cmx, ephemeralKey, encCiphertext
T.4a.ii	ZTxId60ActN_Hash	cv, rk, outCiphertext
T.4a.vi	ZTxIdOrcBurnHash	burn tuples
T.5	ZTxIdSAIssueHash	issuer, issue-actions digest
—	ZTxIdIssuActHash	per action: desc. hash, notes, flags
—	ZTxIdIAcNoteHash	per note: recipient, value, ρ , rseed
T.6	ZTxIdMemo___Hash	nonce, chunks digest
—	ZTxIdMemoCksHash, ZTxIdMemoCk_Hash	memo chunk digests
A.1	ZTxAuthTransHash	scripts, now incl. per-input sighashInfo
A.3	ZTxAuthOrchaHash	group auth digests, bindingSigOrchard
A.3a	ZTxAuthOrcAGHash	proofsOrchard, spend-auth digest
A.3a.ii	ZTxAuthOrSASHash	spendAuthSig encodings
A.4	ZTxAuthZSAOrHash	issueAuthSig encoding

Table 4: ZIP-246 digest personalizations (ZIP-246, TxId Digest and Authorizing Data Commitment). Unnumbered rows are children whose labels the ZIP text assigns inconsistently; see the closing remark of this subsection.

Signature digest. The sighash reuses the tree with the transparent branch replaced by an input-specific variant:

$$\text{sighash} := \text{BLAKE2b-256}(\text{ZcashTxHash}_\parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{transp}}^S \parallel d_{\text{Sap}} \parallel d_{\text{Orch}} \parallel d_{\text{issue}} \parallel d_{\text{memo}}),$$

produced per transparent input, per Sapling input, per OrchardZSA Action, and — new in v6 — per Issue Action; the Sapling, Orchard, issuance, and memo branches are identical to their txid-side counterparts, so the issuance bundle and the memo bundle enter every sighash (ZIP-246, Signature Digest). On this digest ZIP-227 hangs its authorization rule: the SigHash is the §4.10 SIGHASH transaction hash with the ZIP-226 modifications, computed with SIGHASH_ALL, and the issuance authorization signature MUST satisfy

$\text{IssueAuthSig.Validate}(\text{ik}, \text{SigHash}, \text{issueAuthSig}) = 1$, where ik is decoded from the `issuer` field and its encoding is required to begin with `0x00`, the BIP-340 scheme byte (ZIP-227, Specification: Consensus Rule Changes).

Authorizing-data commitment. The commitment to authorizing data follows the same pattern: A.1 covers transparent scripts together with the new per-input `TransparentSighashInfo`; A.2 keeps the ZIP-244 structure with field encodings altered by `sighashInfo`; A.3 commits to the per-group proofs and spend-authorization signatures and to `bindingSigOrchard`; and a new A.4 commits to the `issueAuthSig` field encoding, with a defined empty case (ZIP-246, Authorizing Data Commitment).

Remark 6.1 (The reference text is itself unfinished). ZIP-246 is internally incomplete even as withdrawn text: the authorizing-data section carries “TODO: Update the Authorizing Data Commitment below for the `sighashInfo` changes to the tx format,” Rationale and Reference implementation are TBD, and the subtree labels are inconsistent — `issue_actions_digest` is listed as T.5c but headed T.5a with children T.5a.i, and the burn-digest children are labelled T.4b.i/T.4b.ii under node T.4a.vi (`zip-0246.rst`). Anyone respecifying the format inherits these defects as open editorial work; the implementation, not the ZIP, is currently the more precise artifact.

6.4 Transaction-level consensus rules

One Action Group. Although the format is a vector of Action Groups, ZIP-227 collapses it for OrchardZSA: per transaction, `nActionGroupsOrchard` MUST be 0 or 1, and `nAGExpiryHeight` MUST be 0 (ZIP-227, Specification: Consensus Rule Changes). The vector form is used by draft ZIP-228’s proposed ZSA swaps, which would relax the one-group rule for swap bundles; it is not active in the core ZIP-227 rules.

Issuance requires an Action Group. A transaction with an issuance bundle MUST also contain at least one OrchardZSA Action Group (ZIP-227, Specification: Consensus Rule Changes). The rule is load-bearing twice over. First, issue-note ρ derivation consumes the first Action’s nullifier: each issued note’s ρ is computed by the function ZIP-227 names `DeriveIssuedRho` — constructed in §3.3 — keyed by $\text{nf}_{0,0}$, the nullifier of the input note of the first Action in the first Action Group, over the Issue Action and note indices (ZIP-227, Computation of ρ). The indices domain-separate notes within the transaction, while the nullifier-keyed PRF makes collisions across transactions negligible under its pseudorandomness assumption; it does not give literal global uniqueness. Second, without a spend in the transaction the issuance-only SIGHASH would be replayable; the mandatory Action Group ties the issuance signature to a nullifier that consensus consumes (ZIP-227, Rationale).

Global issuance state. Nodes maintain a map `issued_assets` from asset bases to triples $(\text{balance}, \text{final}, \text{note_ref})$, chained empty map $\rightarrow$ per-block $\rightarrow$ per-transaction. Burns decrement balance, requiring $\text{balance} \geq v$, *before* the transaction’s issuance bundle is processed; issuance requires $\text{final} \neq 1$, increments balance subject to the overflow bound $\text{MAX_ISSUE} = 2^{64} - 1$, and sets final when `finalize = 1`. The first issuance of an asset MUST include a *reference note*: value 0, paid to the default address of the all-zero Orchard spending key (ZIP-227, Global Issuance State and Reference Notes). The 43-byte raw encoding of that address is fixed in the ZIP and reproduced as a constant in the implementation (§6.6).

Burn rules and the extended balance equation. For every $(\text{AssetBase}, v)$ in `assetBurn`: $\text{AssetBase} \neq V^{\text{Orchard}}$ (ZEC cannot be burned this way), $0 < v \leq \text{MAX_BURN_VALUE}$, and no two entries share an `AssetBase`; the bound $\text{MAX_BURN_VALUE} := 2^{63} - 1$ is chosen for compatibility with signed 64-bit value-balance fields (ZIP-226, Burn Mechanism). Burns enter

value balance through the binding key. Where vanilla Orchard computes bvk by subtracting the committed declared balance from the sum of net value commitments (the *Ironwood Guide*, §“The binding signature”), ZIP-226 subtracts one zero-randomness commitment per burn entry as well:

$$\begin{aligned} \text{bvk} := & \left(\sum_{i=1}^n \text{cv}^{\text{net},i} \right) - \text{ValueCommit}_0^{\text{OrchardZSA}}(V^{\text{Orchard}}, v_{\text{balance}}^{\text{Orchard}}) \\ & - \sum_{(\text{AssetBase}, v) \in \text{assetBurn}} \text{ValueCommit}_0^{\text{OrchardZSA}}(\text{AssetBase}, v), \end{aligned}$$

and the prover signs the SIGHASH with $\text{bsk} = \sum_{i=1}^n \text{rcv}_i$ (ZIP-226, Burn Mechanism). Under the unknown-discrete-log assumptions made explicit in §4.3, the signature computationally enforces that every non-ZEC asset nets to exactly its declared burns.

Commitment-tree order. Note commitments enter the global tree in a fixed order: first, per Action Group, the commitments of the new OrchardZSA notes of each Action; then, per Issue Action, the issue-note commitments (ZIP-227, Specification: Consensus Rule Changes). Wallets and nodes must agree on this order to agree on note positions.

6.5 The experimental fee policy

The fee treatment chains through three documents: ZIP-226 (Transaction Fees) defers to ZIP-317 “as described in ZIP 227”; ZIP-227 (Changes to ZIP 317) states that the conventional fee is altered for the presence of issuance actions and the creation of new Custom Assets, pointing at ZIP-317’s Fee calculation section. Fees remain payable in ZEC only (ZIP-227, rationale). All variants share the conventional-fee form documented in the *Wallet Guide*, §“Fees (ZIP-317)”:

$$\text{conventionalFee} := \text{marginalFee} \cdot \max(\text{graceActions}, \text{logicalActions}),$$

with $\text{marginalFee} = 5000$ zatoshis and $\text{graceActions} = 2$ (ZIP-317, Fee calculation).

ZIP 317 contains no ZSA fee contributions. The experimental fork adds a charge for creating an asset, to discourage permanent state growth from unused assets: the parameter $\text{creationCost} = 100$ logical actions, and two contributions summed into logicalActions alongside the base terms of the deployed rule:

$$\begin{aligned} \text{contribution}_{\text{ZSAIssuance}} &:= n_{\text{ZSAIssueNotes}}, \\ \text{contribution}_{\text{ZSACreation}} &:= \text{creationCost} \cdot n_{\text{AssetCreations}}, \end{aligned}$$

where $n_{\text{ZSAIssueNotes}}$ counts issue-note outputs and $n_{\text{AssetCreations}}$ counts Custom Assets newly added to the global issuance state (fork ZIP-317, Fee calculation). The rationale is a one-time charge for state growth: each new asset adds an `issued_assets` row tracked in perpetuity, while subsequent issuance, finalization, and burn only mutate the row. The implementation uses the same contributions. As the *Wallet Guide* states for the upstream rule (§“The conventional fee”), paying the conventional fee is not a consensus requirement in any ZIP-317 variant. Classification: the ZSA fee rule is *fork-specified*; whether it returns upstream as a ZIP-317 revision, as Ironwood’s did, is undecided.

6.6 Implementation status at the pinned commits

Serialization. The experimental reader parses the header, one `vSighashInfo` per transparent input (required to equal the version-0 encoding), the Sapling v6 bundle, the Orchard v6 bundle, and the issuance bundle (`zcash_primitives`, `transaction/mod.rs`). The Orchard reader rejects more than one Action Group (“A V6 transaction must contain at most one action

group”), requires at least one Action, and rejects a nonzero `nAGExpiryHeight`; the writer emits exactly one group with `nAGExpiryHeight = 0` (`transaction/components/orchard.rs`) — the ZIP-227 consensus rules hard-coded at the serialization layer. Issuance-bundle serialization matches ZIP-230, including the two-zero-compactSize empty bundle and rejection of the inconsistent issuer/action combinations. The sighash implementation extends the v5 hash with the issuance digest for ZSA-capable versions, with sighash-info constants `ORCHARD_SIGHASH_INFO_V0 = ISSUE_SIGHASH_INFO_V0 = [0x00]` (`sighash_v6.rs`, `sighash_versioning.rs`). The orchard fork defines the ZIP-246 personalizations of Table 4, asserting the 0x00 BIP-340 byte when hashing the 65-byte issuance signature encoding.

Verification code. Function `derive_bvk_raw` implements the extended balance equation of §6.4 verbatim; `validate_bundle_burn` enforces the ZIP-226 burn rules — non-ZEC asset, non-zero amount, amount $\leq 2^{63} - 1$, no duplicates — plus presence in and sufficient supply under the global issuance state; and `verify_issue_bundle` checks the BIP-340 signature over the 32-byte sighash, the per-note ρ against `DeriveIssuedRho`, non-finalization, u64-checked supply addition, and the reference note on first issuance, whose 43-byte `RAW_REFERENCE_RECIPIENT` constant equals the array in ZIP-227. The 0x84 domain separator for issued-note ρ lives in the QED-it `zcash_spec` fork (rev d5e8426). Proof sizes record the circuit cost: the ZSA aggregated proof has base size 2848 bytes and 2272 bytes per Action, against a vanilla base of 2720 (`primitives/orchard_primitives_zsa.rs`, `orchard_primitives_vanilla.rs`).

Divergences from the withdrawn ZIP text. Four are material. The first three block byte-compatibility with ZIP-230/246 as written; the fourth changes private witness construction but is not enforced by the circuit.

1. *No explicit fee field.* The v6 reader serializes no fee, and `zip233Amount` only behind the `zip-233` cargo feature; the header `txid` digest correspondingly omits ZIP-246’s T.1f fee field, with the comment “TODO: Factor this out ... when implementing ZIP 246 in full” (`transaction/mod.rs`, `txid.rs`).
2. *No memo bundles.* The forks implement the pre-ZIP-231 layout: OrchardZSA notes carry a full 512-byte memo, giving a 612-byte `encCiphertext` with an 84-byte compact prefix (52 vanilla bytes plus the 32-byte asset base), and Sapling v6 outputs are read in the v5 580-byte format (`primitives/zcash_note_encryption_domain.rs`, `transaction/components/sapling.rs`). No memo fields and no T.6 are implemented; instead the action-group `txid` digest inserts a non-spec memos digest (personalization `ZTxIdOrcActMHash`) between the compact and non-compact digests and hashes only the 84-byte compact slice into the compact digest, with TODOs “Remove once new Memo Bundles are implemented (ZIP-231)” (`primitives/orchard_primitives_zsa.rs`).
3. *Lead byte.* The implementation hard-codes `NOTE_VERSION_BYTE_V3 = 0x03`; upstream ZIP-230 withdrew that claim after ZIP-2005 took 0x03, leaving the ZSA lead byte an unassigned placeholder. The fork’s choice predates upstream ZIP-2005 assigning 0x03 to Ironwood notes and directing future proposals to pick a new value; the collision is fork-only context, with no format ambiguity on Zcash mainnet. Reassignment is a hard prerequisite for any deployment: the lead byte selects the note-plaintext parser.
4. *Split-note ψ^{nf} .* Upstream ZIP-226 now derives the split-note nullifier randomizer as $\psi^{\text{nf}} = \text{ToBase}(\text{PRFexpand}_{\text{rseed}_{\text{nf}}}(\text{0x0A} \parallel \rho^{\text{old}}))$ with `rseednf` sampled uniformly at random, the first-byte domain 0x0A “reserved by ZIP 2005 for this use” (ZIP-226, split-note nullifier); the fork at cf801a5d realizes the older fork text (uniformly random ψ^{nf}) via a separate `rseed_split_note` under the ordinary ψ domain 0x09 (`note.rs`, `circuit.rs`).

6.7 Wallet integration

The withdrawn ZIP-230 and ZIP-226 design imposes two obligations. First, all wallets SHOULD switch to sending only its v6 transactions once permitted: a v5 transaction reveals that all its Orchard notes are ZEC, and the design’s v6 OrchardZSA output format incorporates the quantum-recoverability changes (ZIP-230, Implications for Wallets; ZIP-226, Backwards Compatibility with ZEC Notes). This is a claim about the withdrawn carrier, not the different v6 format now assigned to ZIP-229. Second, wallets MUST support parsing and trial-decrypting v6 outputs — the new lead byte and the memo-bundle changes — by activation, or rescan once support lands: funds sent to an un-upgraded wallet are temporarily inaccessible, because the new lead byte selects different rcm/ψ derivations and a v5-decryption wallet rejects the note (ZIP-230, Implications for Wallets). Payment requests have a specified extension path: ZIP-321’s `req-asset` parameter carries an Asset Identifier and amount (the *Wallet Guide*, §“The URI grammar”).

Multi-party and hardware signing is the open end. No PCZT structure exists for OrchardZSA bundles or issuance bundles: the `librustzcash` fork’s transaction extractor panics with “PCZT support for ZSA is not implemented” when handed an OrchardZSA bundle, the orchard fork’s PCZT module is written against the vanilla flavor only, and the released PCZT v2 targets ZIP-229’s Ironwood v6, not ZSA (the *Wallet Guide*, §“Partially created transactions (PCZT)” and §“PCZT v2: deferred anchors and the Ironwood bundle”). Classification: *open* — hardware-wallet and multi-party signing for ZSAs is unspecified in any draft.

6.8 Outlook

Between the pinned implementation and any deployment stand three cross-cutting respecification problems.

1. *A carrier.* Transaction version 6 now belongs to NU6.3’s Ironwood format (ZIP-229), whose non-requirements exclude the ZSA machinery (§6.1). The designated successor for what ZIP-230/246 deferred — an extensible transaction format — is ZIP-248, now a merged Draft for V7. Its registry lists OrchardZSA with a provisional variant and ZSA issuance without an allocated bundle type, under “Potential Future Bundle Types”; it does not specify either bundle’s encoding or digest. The ZSA-specific transaction format therefore remains incomplete: before a future ZSA deployment, the byte format of §6.2 must be mapped or respecified for V7, and §6.3 re-anchored to the resulting digest rules.
2. *A base protocol.* ZIP 226 assumes ZIP 2005’s recoverability changes, but the experimental implementation does not. No ZIP specifies whether OrchardZSA targets the restricted Orchard pool or is rebased onto Ironwood. Classification: *open*.
3. *Reconvergence of spec and code.* The divergence list of §6.6 — fee field, memo bundles, lead byte, ψ^{nf} — plus the orphaned fee treatment of §6.5 must land on one side or the other: either the respecified ZIPs adopt what the code does, or the code moves. Each divergence is identified, but resolving it still requires protocol choices and implementation work.

The summary judgment this volume can defend: a ZSA transaction layer is designed and implemented in the pinned forks, but the deployable protocol is *currently unspecified* in the sense this series requires — there is no complete merged carrier specification under which a mainnet node could implement it. The byte-level facts of this section are fixed by withdrawn ZIP text and pinned fork code, and they will remain the diff base for any respecification; the remaining open items are enumerated above and in the scope section.